Campus Cat Support Service

Catpus

Introduction to Software Engineering [SWE3002_42]

Team 2

Team Leader	2020315545 **박진석**
Member	2021311693 **박보연**
Member	2019313301 **박지수**
Member	2020310960 **오정호**
Member	2020310452 **조현우**
Member	2020314853 **최민성**

Table of Contents

1. Introduction 11

1.1 Purpose 11

1.2 Definitions.....11

1.3 Intended Audience and Reading Suggestions 14

1.4 Product Scope 14

1.5 References.....16

2. Overall Description 17

2.1 Product Perspective 17

2.1.1 Market Status 17

2.2 Product Functions.....18

2.2.1 Create a User Account and Login 18

2.2.1.1 Sign-up 18

2.2.1.2 Login 18

2.2.1.3 Reset Password 18

2.2.1.4 Profile Setting 18

2.2.2 Cat Care Information and Activity 19

2.2.2.1 View Feeding Locations 19

2.2.2.2 Participate in Volunteer Activities 19

2.2.3 Pet Sitting Matching System 19

2.2.3.1 Request a Pet Sitter 19

2.2.3.2 Accept or Decline a Request 19

2.2.3.3 Matching History 19

2.2.4 Review and Trust Level System 20

2.2.4.1 Write a Review 20

2.2.4.2 Trust Level Calculation 20

2.2.5 Badge and Reward System 20

2.2.5.1 Badge Collection 20

2.2.5.2 My Badges 20

2.2.6 SNS Sharing and Community 20

2.2.6.1 Cat-specific Community Boards 20

2.2.6.2 My Petstagram 21

2.3 User Classes and Characteristics.....21

2.3.1 General Users 21

2.3.2 Active Users 21

2.3.3 System Manager 22

2.4 Operating Environment 22

2.4.1 Hardware 22

2.4.2 Software 22

2.5 Design and Implementation Constraints.....22

2.6 User Documentation.....23

2.7 Assumptions and Dependencies 23

3. Specific Requirements 24

3.1 External Interface Requirements 24

3.1.1 User Interfaces 24

3.1.1.1 화면 배치 제약 사항 41

3.1.1.2 표준버튼 및 기능 42

3.1.1.3 오류 메시지 표준 42

3.1.2 Hardware Interfaces 43

3.1.2.1 지원 장치 유형 43

3.1.2.2 데이터 및 제어 상호작용 43

3.1.2.3 통신 프로토콜 44

3.1.2.4 전력 관리 및 최적화 44

3.1.2.5 보안 및 개인정보 보호 45

3.1.3 Software Interfaces 45

3.1.3.1 백엔드 서버 45

3.1.3.2 데이터 흐름 및 메시지 형식 46

3.1.3.3 서비스와 통신 방식 47

3.1.3.4 데이터 공유 메커니즘 48

3.1.4 Communications Interfaces 48

3.1.4.1 Email 48

3.1.4.2 Web Browser 48

3.1.4.3 Network Server Protocol 49

3.1.4.4 Communication Standard 49

3.1.4.5 Message Format 49

3.1.4.6 Security and Encryption Issues 50

3.1.4.7 Data Transfer Speed and Synchronization Mechanism 51

3.2 Functional Requirements 51

3.2.1 User 51

3.2.2 Auth 52

3.2.3 Cat Care and Volunteer Activities 54

3.2.4 Pet Sitting Matching System 55

3.2.5 Review and Trust Level System 57

3.2.6 Badge and Reward System 58

3.2.7 SNS Sharing and Community 60

3.2.8 Use Case Diagrams 61

3.3 Nonfunctional Requirements 62

3.3.1 Performance Requirements 62

3.3.2 Safety Requirements 62

3.3.3 Security Requirements 63

3.3.4 Software Quality Attributes 63

3.3.5 Business Rules 64

3.3.6 Community Guideline 65

3.3.6.1 가이드라인 목적 65

3.3.6.2 콘텐츠 관련 규정 65

3.3.6.3 사용자 에티켓 66

3.3.6.4 동물 복지 및 윤리 66

3.3.6.5 개인정보 보호 66

3.3.6.6 지적재산권 66

3.3.6.7 가이드라인 위반 시 조치 67

3.3.6.8 신고 및 모니터링 67

3.3.6.9 가이드라인의 업데이트 67

3.4 Other Requirements 68

3.4.1 Database Requirements 68

3.4.1.1 데이터베이스 선택 및 구성 68

3.4.1.2 ER Diagram 68

3.4.1.3 데이터 모델 및 컬렉션 구조 69

3.4.1.4 인덱싱 전략 79

3.4.1.5 데이터 제약 조건 79

3.4.2 System Architecture 80

3.4.2.1 사용자 인증 및 계정 관리 80

3.4.2.2 급식소 관리 및 위치 서비스 81

3.4.2.3 펫시팅 매칭 시스템 81

3.4.3 Organization Requirements 81

3.4.3.1 Environmental Requirements 81

3.4.3.2 Development Requirements 81

3.4.3.3 Regulatory Requirements 82

3.4.3.4 Context Model 83

4. Appendix 84

4.1 Document History 84

List of Figures

[Figure 1] User Interface - Login.....24

[Figure 2] User Interface - Sign-up.....26

[Figure 3] User Interface - Home..... 28

[Figure 4] User Interface - Cat Feeding Station.....30

[Figure 5] User Interface - Activity Participation 32

[Figure 6] User Interface - Pet Sitting Request 34

[Figure 7] User Interface - Pet Sitting Matching 36

[Figure 8] User Interface - My Petstagram 38

[Figure 9] User Interface - Settings 40

[Figure 10] Use Case Diagrams.....61

[Figure 11] MongoDB ERD. 68

[Figure 12] System Architecture.....80

[Figure 13] Context Model.....83

List of Tables

[Table 1] Table of General Terms.....12

[Table 2] Table of Project-Specific Terms.....12

[Table 3] Table of Technical Terms.....13

[Table 4] Table of User Interfaces.....24-41

[Table 4-1] User Interface – Login Interface.....24

[Table 4-2] User Interface – Sign-Up Interface.....26

[Table 4-3] User Interface – Home Interface.....28

[Table 4-4] User Interface – Cat Feeding Station Interface.....30

[Table 4-5] User Interface – Activity Participation Interface.....32

[Table 4-6] User Interface – Pet Sitting Request Interface.....34

[Table 4-7] User Interface – Pet Sitting Matching Interface.....36

[Table 4-8] User Interface – My Petstagram Interface.....38

[Table 4-9] User Interface – Settings Interface.....40

[Table 5] Table of Minimum Hardware Specifications.....43

[Table 6] Table of Use Cases.....51-61

[Table 6-1] Use Case of Sign-In.....51

[Table 6-2] Use Case of Profile Settings.....52

[Table 6-3] Use Case of Login/Logout.....52

[Table 6-4] Use Case of Access Token Refresh.....53

[Table 6-5] Use Case of Cat Feeding Station Locations.....54

[Table 6-6] Use Case of Join Volunteering.....54

[Table 6-7] Use Case of Request Pet Sitting.....55

[Table 6-8] Use Case of Accept/Decline Request.....56

[Table 6-9] Use Case of View Matches.....56

[Table 6-10] Use Case of Write Review.....57

[Table 6-11] Use Case of Trust Level Calculation.....58

[Table 6-12] Use Case of Earn Badge.....58

[Table 6-13] Use Case of View Badges.....59

[Table 6-14] Use Case of Cat Community.....60

[Table 6-15] Use Case of Petstagram.....60

[Table 7] Table of MongoDB Collection Information.....70

[Table 8] Table of Document History.....84

1. Introduction

1.1 Purpose

본 문서는 성균관대학교 소프트웨어공학 프로젝트 2팀이 개발하는 캠퍼스 기반 길고양이 및 반려동물 돌봄 커뮤니티 플랫폼인 Catpus의 소프트웨어 요구사항 명세서(SRS)이다. Catpus는 집을 떠나 생활하는 대학생들과 보호와 돌봄이 필요한 동물들을 연결함으로써, 인간과 동물 모두에게 정서적 유대감 형성과 조화로운 공존의 환경을 제공하는 것을 목표로 한다.

기존의 캠퍼스 길고양이 돌봄 활동은 주로 학생들의 자발적인 참여에 의존하고 있었으며, 체계적인 관리 시스템의 부재로 인해 안전성과 지속 가능성에 한계가 있었다. 또한 상업적 펫시팅에 방식 또한 길고양이들에게 적용할 수 없는 한계가 있다. 이에 본 플랫폼은 길고양이 급식소 운영, 돌봄 활동 인증, 대리 산책/돌봄 연결 및 커뮤니티 기능을 포함한 통합 시스템을 제공함으로써, 동물 보호 활동을 보다 투명하고 신뢰성 있게 운영할 수 있도록 한다. Catpus는 이러한 기능을 통해 학생들과 동물이 함께 어우러질 수 있는 지속 가능한 캠퍼스 돌봄 문화를 구축하는 데 기여하고자 한다.

이 애플리케이션은 버전 1.0을 기준으로 작성되었으며, 플랫폼의 전체 기능과 사용자 요구사항을 포괄적으로 정의한다. 본 명세서는 Catpus의 요구사항을 체계적으로 분석 및 기술한 문서로, 이후 Catpus의 설계와 구현은 이 문서를 기반으로 진행될 예정이다. Catpus의 주요 사용자는 반려동물을 키우거나 동물 돌봄에 관심이 있는 성균관대학교 학생들로, 본 SRS는 이들과 관련된 요구사항을 중점적으로 다룬다. 또한, 본 명세서는 소프트웨어공학 프로젝트 2팀의 개발 전 과정에서 활용될 예정이며, 개발자, 디자이너, 기획자 등 프로젝트 팀 전원이 일관된 시스템 이해를 공유하고, 설계·구현·테스트 및 유지보수의 기준 문서로서 기능할 것이다.

1.2 Definitions

[Table 1] Table of General Terms

Term	Definition / Meaning
SRS	Software Requirements Specification의 약자로, 소프트웨어 요구사항 명세서를 의미한다.
Catpus	“Cat”과 “Campus”의 합성어로, 본 프로젝트의 애플리케이션 이름이다.
UI	User Interface의 약자로, 사용자가 시스템과 상호작용하는 화면이나 컴포넌트를 의미한다.
UX	User Experience의 약자로, 사용자가 시스템을 사용하면서 느끼는 전반적인 경험을 의미한다.
API	Application Programming Interface의 약자로, 다른 소프트웨어 구성 요소와 통신하기 위한 인터페이스이다.

[Table 2] Table of Project-Specific Terms

Term	Definition / Meaning
급식소	캠퍼스 내에 설치된 길고양이를 위한 사료와 물을 제공하는 공간으로, 자원봉사자들이 관리한다.
펫시팅 (Pet Sitting)	반려동물 주인이 부재 시 대신 반려동물을 돌봐주는 활동을 의미한다.
신뢰도 레벨 (Trust Level)	사용자의 활동 이력·리뷰·커뮤니티 참여도 등을 종합하여 계산된 신뢰 수준으로, 서비스 이용 권한과 직결된다.
Petstagram	Instagram과 Pet의 합성어로, 애플리케이션 내에서 반려동물 사진과 일상을 공유하는 SNS 기능이다.

배지 (Badge)	특정 활동이나 기여를 완료한 사용자에게 부여되는 디지털 업적 표시로, 사용자의 프로필에 표시된다.
지오펜싱 (Geofencing)	특정 지리적 영역을 가상으로 설정하고, 사용자가 해당 영역에 들어가거나 나갈 때 알림을 제공하는 기술이다.
활동 인증	봉사 활동이나 펫시팅 활동이 실제로 이루어졌음을 증명하기 위한 사진 등의 증거 제출 과정이다.

[Table 3] Table of Technical Terms

Term	Definition / Meaning
MongoDB	문서 지향적 NoSQL 데이터베이스로, JSON 형태의 문서를 저장하고 관리한다.
JWT	JSON Web Token의 약자로, 사용자 인증 및 정보 교환을 위한 개방형 표준이다.
REST API	Representational State Transfer의 원칙을 따르는 API로, HTTP 프로토콜을 기반으로 리소스에 접근하는 방식이다.
WebSocket	클라이언트와 서버 간의 양방향 통신을 지원하는 프로토콜이다.
AES-256	Advanced Encryption Standard의 약자로, 256비트 키 길이를 사용하는 대칭 암호화 알고리즘이다.
TLS 1.3	Transport Layer Security의 최신 버전으로, 인터넷 통신의 보안을 위한 암호화 프로토콜이다.

1.3 Intended Audience and Reading Suggestions

이 문서는 Catpus 프로젝트에 참여하거나 해당 시스템에 관심이 있는 다음과 같은 독자들을 대상으로 한다:

- **개발자**: 2장, 3장, 4장의 기능 요구사항 및 시스템 아키텍처를 중심으로 참조
- **기획자 및 관리자**: 1장과 2장을 통해 전체 시스템 개요 및 기능 흐름 파악
- **디자이너**: 사용자 인터페이스 요구사항 및 커뮤니티 기능 설계에 참고
- **테스터 및 유지보수 담당자**: 4장과 5장의 상세 요구사항 및 비기능 요구사항 참조

문서는 다음의 순서로 읽는 것을 권장한다:

1. **1장**: 프로젝트 개요 및 목적 파악
2. **2장**: 시스템의 전반적인 구조와 기능 개요
3. **3장~4장**: 상세 요구사항 및 시스템 기능
4. **5장 이후**: 비기능 요구사항 및 기타 고려 사항

1.4 Product Scope

Catpus는 성균관대학교 자연과학캠퍼스를 중심으로, 교내에서 이루어지는 길고양이 돌봄 활동과 반려동물 대리 산책/돌봄 서비스를 하나의 통합 커뮤니티 플랫폼으로 연결하는 것을 목표로 한 모바일 애플리케이션이다. 본 플랫폼은 비공식적인 오프라인 중심의 돌봄 활동이 가지는 정보 단절, 관리 부재, 신뢰 부족 등의 문제를 해결하고, 학생 주도의 자율적이고 지속 가능한 돌봄 문화를 구축하고자 기획되었다.

Catpus는 사용자가 돌봄 제공자 또는 의뢰자로 참여할 수 있는 양방향 구조를 가지고 있으며, 다음과 같은 주요 기능들을 제공한다:

- 고양이 돌봄 정보 공유 및 봉사 활동 참여 기능: 교내에 설치된 고양이 급식소의 위치를 지도 기반으로 확인하고, 자발적인 봉사활동(급식소 관리, 보호소 정리 등)에 참여할 수 있도록 한다.
- 펫시팅 매칭 시스템: 보호자가 애완동물의 산책 혹은 임시 보호를 요청하면, 일정 기준 이상의 기여도를 달성한 검증된 사용자(펫시터)가 이를 수락할 수 있는 매칭 기능을 제공한다. 신뢰도 기반 제한으로 안전성과 책임성을 확보한다.
- 리뷰 및 신뢰도 평가 시스템: 활동 종료 후 사용자 간의 리뷰 및 피드백 작성을 통해 신뢰 점수를 계산하며, 활동 이력 및 신고 여부 등을 반영하여 자동으로 Trust Level을 관리한다.
- 배지 및 보상 시스템: 특정 기준을 달성한 사용자에게 배지를 부여하며, 이는 향후 매칭 활동에서의 신뢰 지표로 활용된다.
- 고양이 전용 커뮤니티 및 Petstagram: 교내 길고양이 별 게시판을 통해 목격 정보, 건강 상태 등의 내용을 공유하고, 사용자가 자신의 반려동물 일상도 공유할 수 있도록 하여 동물 중심의 소셜 커뮤니케이션 환경을 조성한다.

Catpus는 단순히 기능적 서비스를 제공하는 수준을 넘어서, 학생과 동물의 공존을 실현하는 공동체적 가치를 실현하는 데에 중점을 두고 있다. 이 플랫폼은 다음과 같은 전략적 목표와 효과를 기대한다:

- 정서적 유대감 증진: 가족과 떨어져 생활하는 대학생들에게 동물과의 교류를 통해 심리적 안정을 제공한다.
- 공동체 기반의 신뢰 문화 형성: 단발성 서비스 매칭이 아닌, 활동 이력과 기여도를 기반으로 한 지속적이고 검증 가능한 신뢰 시스템을 구축한다.
- 동물 보호 활동의 체계화 및 접근성 향상: 기존의 비공식 돌봄 활동을 애플리케이션 기반으로 구조화하여, 더 많은 학생들이 쉽게 참여하고, 책임감 있는 보호활동을 실현할 수 있도록 한다.
- 비용 부담 없는 상호 돌봄 문화 확산: 상업적 펫시팅 서비스가 가지는 금전적 장벽을 제거하고, 학생 간 자발적 교류를 기반으로 하는 비영리 돌봄 모델을 구현한다.

성공 지표는 다음과 같다.

- 월간 활성 사용자(MAU) ≥ 5 000명
- 펫시팅·봉사 매칭 성공률 ≥ 80 % (30일 기준)

- 급식소 봉사 참여 인원 ≥ 300명/학기
- 활동 완료 후 긍정 리뷰 비율 ≥ 90 %

본 애플리케이션은 독립형 모바일 앱으로 개발되며, 향후에는 성균관대학교 외 타 캠퍼스, 지역 동물 보호 단체 등과의 연계를 통해 서비스 범위 확장도 고려한다. 이러한 구조를 통해 Catpus는 기여와 책임, 돌봄과 신뢰가 선순환하는 캠퍼스 커뮤니티를 지향하며, 동물과 사람이 조화롭게 공존할 수 있는 지속 가능한 디지털 생태계의 초석을 마련하고자 한다.

1.5 References

1.5.1. IEEE 표준 830 에 대한 가이드 IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications, In IEEEXplore Digital Library http://ieeexplore.ieee.org/Xplore/guesthome.jsp

1.5.2. SRS 우수사례 “Software Requirement Specification”. SKKU 2021 Introduction to Software Engineering Team 1, 2021 (참고한 SRS 우수사례, Highlighting SKKU online campus life) https://github.com/skkuse/2021spring_41class_team1

1.5.3. SRS 우수사례 “Software Requirement Specification”. SKKU 2021 Introduction to Software Engineering Team 7, 2021 (참고한 SRS 우수사례, Finding Friends for the Same Purpose at Close Distance) https://github.com/skkuse/2021spring_41class_team7

2. Overall Description

2.1 Product Perspective

Catpus는 성균관대학교 자연과학캠퍼스 내에서 길고양이 돌봄 활동 및 반려동물 돌봄 서비스를 연결하고자 하는 학생들을 위한 애플리케이션이다. 본 애플리케이션은 기존 오프라인 중심의 비공식 활동 방식과 상업적 펫시팅(pet sitting: 반려동물을 대신 돌봐주는 행위) 서비스가 가지는 한계점을 고려하여, 자발적 돌봄 온라인 커뮤니티 플랫폼을 구축하고자 기획되었다.

사용자는 활동 목적에 따라 돌봄 제공자 또는 의뢰자로 애플리케이션을 사용할 수 있으며, 레벨 시스템, 후기 기반 신뢰도 평가, 활동 이력 관리, 배지 시스템, SNS 공유 기능 등을 통해 지속적이고 활발한 참여를 유도한다. 일정 기준 이상의 활동을 수행한 사용자만 펫시팅 매칭에 참여할 수 있도록 설계하여 플랫폼 내 신뢰를 강화한다. 이 애플리케이션은 독립형 모바일 앱으로 운영되며, 추후 다양한 보호 커뮤니티나 타 캠퍼스와의 연계를 통해 확장될 수 있다.

2.1.1 Market Status

현재 대부분의 상용 펫시팅 서비스는 금전적 비용을 기반으로 운영되며, 이는 대학생 사용자들에게 비용적인 부담을 야기해 실질적인 이용률을 낮추는 요인으로 작용한다. 또한, 교내 길고양이 보호 활동은 주로 비공식적인 커뮤니티나 자발적 개인의 노력에 의해 유지되고 있다. 이러한 환경에서는 정보의 지속성, 접근성, 신뢰성 확보가 어려워 체계적인 돌봄 활동이 어렵고, 참여자 간 신뢰 형성도 제한적이다. 또한 보호자가 펫시팅을 요청할 경우, 해당 인물에 대한 이력이나 후기 없이 단발성 매칭이 이루어져 안정성과 연속성 측면에서 한계가 발생한다.

이러한 한계를 극복하기 위해 본 애플리케이션은 캠퍼스 안에서 학생들이 신뢰를 바탕으로 자발적인 돌봄 활동에 참여할 수 있는 구조를 만들어, 책임감 있는 돌봄 문화를 형성하고자 한다.

2.2 Product Functions

본 애플리케이션은 캠퍼스 내 길고양이 돌봄과 반려동물 펫시팅 활동을 효율적으로 연결하고, 사용자 간의 신뢰 기반 상호작용을 지원하는 다양한 기능을 제공한다.

2.2.1 Create a User Account and Login

본 서비스를 이용하기 위한 첫번째 단계로, 사용자는 회원가입 절차를 거쳐 서비스를 시작한다. 로그인한 사용자에게는 활동 참여, 펫시팅 매칭, 후기 작성 등의 주요 기능이 제공된다.

2.2.1.1 Sign-up

회원가입 시 사용자는 이메일 주소, 비밀번호, 이름(닉네임), 활동 목적(펫시터 / 보호자 / 봉사자)을 입력한다. 이후 본인 이메일로 전송된 인증번호를 입력하면 가입이 완료된다.

2.2.1.2 Login

기존 회원은 이메일과 비밀번호를 입력해 로그인할 수 있으며, 로그인 유지 상태에서만 활동 참여가 가능하다. Google이나 Apple로도 로그인할 수 있다.

2.2.1.3 Reset Password

비밀번호를 분실한 경우, 가입 시 등록한 이메일을 통해 인증번호를 받고 비밀번호를 재설정할 수 있다.

2.2.1.4 Profile Setting

회원가입 이후 사용자는 프로필 이미지, 닉네임, 관심 동물(고양이, 강아지 등), 자기소개 등을 설정할 수 있으며, 이 정보는 매칭 및 활동 참여 시 노출된다.

2.2.2 Cat Care Information and Activity

사용자는 애플리케이션을 통해 교내 길고양이 급식소 위치, 보호소 정보, 자율 봉사 활동 내역 등을 확인할 수 있다.

2.2.2.1 View Feeding Locations

지도 기반 인터페이스를 통해 캠퍼스 내 고양이 급식소 위치를 시각적으로 확인할 수 있다.

2.2.2.2 Participate in Volunteer Activities

사용자는 급식소 관리, 고양이 물품 정리 등 교내 고양이 돌봄 활동 뿐만 아니라 근처 동물 보호소 봉사 활동에 참여 신청을 할 수 있으며, 참여 시 활동 내역이 기록된다.

2.2.3 Pet Sitting Matching System

앱의 핵심 기능 중 하나인 펫시팅 매칭은 일정 기준 이상의 활동 레벨을 달성한 사용자만 접근 가능하다.

2.2.3.1 Request a Pet Sitter

반려동물 보호자는 날짜, 시간, 종류, 요청사항 등을 입력하여 펫시터 요청을 등록할 수 있다.

2.2.3.2 Accept or Decline a Request

펫시터는 본인의 일정과 조건에 따라 요청을 수락하거나 거절할 수 있다. 매칭이 성사되면 채팅 기능이 활성화된다.

2.2.3.3 Matching History

과거 펫시팅 내역은 사용자 프로필에 저장되며, 후기 작성 및 활동 점수 반영에 활용된다.

2.2.4 Review and Trust Level System

2.2.4.1 Write a Review

매칭 종류 후, 사용자들은 서로에 대한 리뷰(별점 및 간단한 피드백)를 작성할 수 있다.

2.2.4.2 Trust Level Calculation

리뷰, 누적 활동 수, 신고 여부 등을 기반으로 시스템은 자동으로 사용자 신뢰 레벨을 계산한다.

2.2.5 Badge and Reward System

2.2.5.1 Badge Collection

사용자는 일정 기준(예: 고양이 돌봄 활동 10회, 펫시팅 3건 이상)을 달성한 경우 특정 배지를 획득할 수 있다.

2.2.5.2 My Badges

획득한 배지는 프로필에 표시되며, 향후 펫시팅 요청 시 상대방이 참고할 수 있는 신뢰 지표가 된다.

2.2.6 SNS Sharing and Community

애플리케이션은 사용자 간의 정보 공유와 소통을 위한 커뮤니티 기능을 제공한다.

2.2.6.1 Cat-specific Community Boards

교내에 거주하는 고양이마다 별도로 구성된 게시판에서, 사용자는 해당 고양이의 사진, 위치, 건강 상태, 최근 목격 정보 등을 등록하고 확인할 수 있다.

2.2.6.2 My Petstagram

사용자는 자신의 반려동물 사진, 일상, 귀여운 모습 등을 반려동물 전용 게시판에 글을 올릴 수 있다. 다른 사용자들과 댓글, 좋아요 등으로 소통이 가능하며, 자연스럽게 캠퍼스 내 반려동물 커뮤니티 문화를 형성할 수 있다.

2.3 User Classes and Characteristics

본 애플리케이션의 주요 사용자는 성균관대학교 자연과학캠퍼스를 기반으로 길고양이 돌봄 활동이나 반려동물 펫시팅에 참여하고자 하는 학생, 교직원 등 교내 구성원들이다. 사용자 클래스는 사용 빈도, 기능 사용범위, 역할과 책임에 따라 크게 세가지로 구분된다.

2.3.1 General Users

일반 사용자는 애플리케이션의 기본 기능을 사용하는 성균관대학교의 학생 및 교직원으로 구성된다. 이들은 고양이 급식소 위치 확인, 커뮤니티 게시판 열람, 펫시팅 요청 등의 기능을 간헐적으로 이용하는 사용자들이다. 일반 사용자는 기본적인 한국어 읽기 및 이해 능력을 갖추고 있으며, 스마트폰에 애플리케이션을 설치하고 사용할 수 있는 수준의 디지털 활용 능력을 지니고 있다고 가정한다. 또한, 커뮤니티 규칙을 인지하고 준수하며, 활동 종료 후 간단한 후기를 작성할 수 있는 사용자로 기대된다.

2.3.2 Active Users

활동 사용자는 고양이 돌봄 봉사나 펫시팅 제공자로 적극 참여하는 사용자로, 시스템의 핵심 기여자 역할을 수행한다. 이들은 비교적 높은 빈도로 애플리케이션을 사용하며, 자발적이고 책임감 있는 태도로 돌봄 활동에 참여한다. 활동 사용자는 일정 수준 이상의 활동 이력과 후기를 통해 신뢰 기반의 레벨을 획득해야 하며, 해당 레벨을 기준으로 펫시팅 매칭 기능 사용이 가능해진다. 또한 고양이나 반려동물에 대한 기초적인 돌봄 지식과 응급상황 발생 시 대처할 수 있는 판단력을 갖춘 것이 바람직하다. 이 사용자들은 커뮤니티 활성화, 시스템 신뢰도 형성 등에서 중요한 역할을 담당하기 때문에, 본 애플리케이션의 주요 요구사항은 이들의 참여와 만족을 최우선으로 고려하여 설계된다.

2.3.3 System Manager

시스템 관리자는 애플리케이션의 운영을 담당하는 내부 관리자 또는 개발자로, 전반적인 시스템 구조와 기능 흐름을 숙지하고 있어야 한다. 이들은 사용자 신고 대응, 게시물 검토, 사용자 제재 등의 권한을 갖고 있으며, 공정하고 윤리적인 기준에 따라 커뮤니티를 유지할 책임이 있다. 또한 시스템 오류나 악용 사례가 발생했을 경우 이를 신속하게 파악하고 해결할 수 있어야 하며, 업데이트나 정책 변경 사항을 반영하는 능력도 필요하다. 시스템 관리자는 일반 사용자와는 달리 높은 수준의 기술적 이해와 운영 경험을 요구받는다.

2.4 Operating Environment

2.4.1 Hardware

본 애플리케이션은 Android 및 iOS 운영 체제를 기반으로 하여 mobile phone을 위해 개발되었다. Catpus가 구동되는 사용자 기기는 최소 2GB RAM 용량과 듀얼 코어1.5 GHz 이상의 프로세서, Wi-Fi 802.11 a/b/g/n/ac, 4G LTE, 5G 이상의 환경 기기에 내장되어 있음을 전제한다.

2.4.2 Software

해당 어플리케이션은 최소 Android OS 11.0(API 레벨 30) 혹은 iOS 18.3 이상의 버전을 전제하고 있다. 따라서, 더욱 낮은 사양의 경우 Catpus의 모든 서비스를 기획한대로 원활하게 이용할 수 없다.

2.5 Design and Implementation Constraints

Catpus 애플리케이션의 설계 및 구현은 다음과 같은 제약 사항을 고려하여 Implementation될 것이다. 아래는 구체적인 Implementation 방향에 대한 방향이 기술되어 있다.

- 시스템은 사용자의 편의와 요구사항을 고려해야 하며, 해당 내용에 따라 수정 가능하다.
- 시스템은 3GB 이상인 RAM 을 가진 모바일 디바이스에서 실행되어야 한다.

- 설치와 구동을 취해 최소 100MB 의 여유공간이 디바이스에 존재해야 한다
- 어떤 플랫폼에서 해당 앱으로 접속하더라도 항상 동일한 기능을 제공해야 한다.
- 시스템은 듀얼코어 1.5 GHz 이상인 프로세서를 가진 모바일 디바이스에서 실행되어야한다.
- 기본적으로 한국어를 지원한다.
- 시스템을 사용하기 위해, 사진 등록을 위한 갤러리 접근, 위치 인증을 위한 위치 정보 제공에 대한 허용 권한을 요구할 수 있으며, 권한 허용이 되지 않을 시 서비스에 이용에 차질이 발생할 수 있다.
- 개인정보 보호 및 데이터 보안이 철저히 이루어져야 하며, 모든 사용자 데이터는 암호화되어 관리되어야 한다.
- 최소 Python 3.7 버전 이상, Java 17 버전 이상, Lotlin 1.9 버전 이상 Django 2.2.5 버전을 이용해 개발한다.
- 최소 Android OS 12.0(API 레벨 31) 혹은 iOS 15.0으로 개발한다.
- Android version 12.0(API 레벨 31)을 사용하여 시스템을 Emulate 한다.

2.6 User Documentation

사용자가 애플리케이션 및 서비스를 이용할 수 있도록 지원하기 위해서 몇 가지 문서가 필요하다. 첫째, 최소 하드웨어 및 소프트웨어 요구사항을 다룬 문서이다. 둘째, 사용자 매뉴얼이 필요하다. 사용자 매뉴얼의 내용은 주요 기능에 대한 상세 설명과 사용 방법이 단계적으로 기술된 문서이다. 회원가입 및 급식소 현황 파악, 펫시팅 매칭, 봉사 신청 등의 특징적인 기능에 대해서는 서비스 이용 전과정에 대한 스크린 샷 및 설명 영상을 함께 제공한다. 마지막으로, 시스템 개발자와 연락할 수 있는 정보를 제공한다.

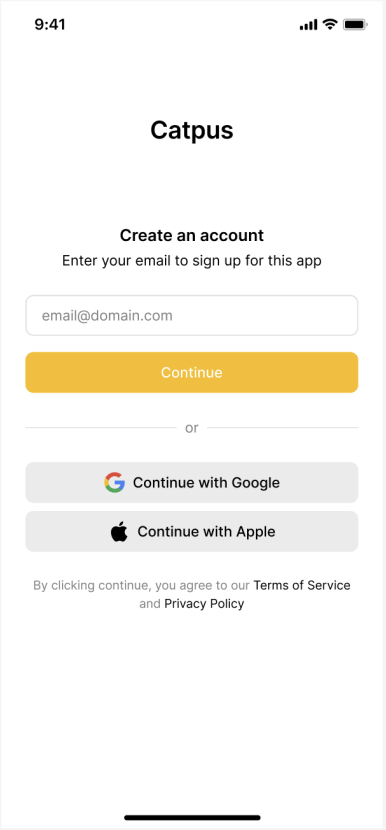
2.7 Assumptions and Dependencies

해당 애플리케이션은 기본적으로 Android 12.0이상, iOS 15.0 이상의 device를 바탕으로 설계 및 구현되었다. 따라서 모든 기능과 컨텐츠는 해당하는 두 운영체제 내에서 정상적으로 작동한다.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

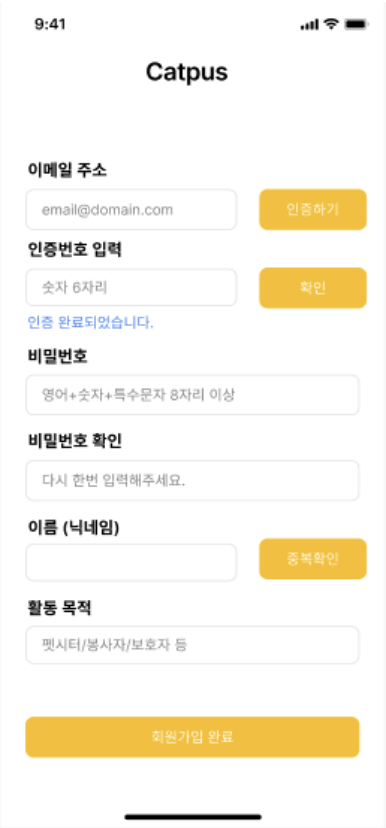


[Figure 1] User Interface - Login

[Table 4-1] User Interface - Login Interface

Name	로그인 화면
Purpose/Description	사용자가 기존 계정으로 Catpus 애플리케이션에 접근하기 위해 이메일

	또는 외부 계정을 통해 로그인할 수 있는 화면이다.
Input source/ Output destination	Client/Server
Unit	Screen
Time/Velocity	-
Relationship with other input/outputs	이메일 주소, 비밀번호 / 로그인 요청, 인증 토큰 / 로그인 실패 메시지
Format,Configuration of screen & User Flow	1. 사용자가 이메일 주소를 입력한다. 2. Continue 버튼을 눌러 서버로 로그인 요청을 전송한다. 3. 사용자가 Google 또는 Apple 계정을 통해 로그인할 수도 있다. 4. 로그인 성공 시 홈 화면으로 이동한다. 5. 약관 및 개인정보 처리방침 안내 문구가 하단에 표시된다.
Format, Configuration of window	● 중앙 정렬된 단일 열 UI 구성 ● 상단 앱 로고 -> 입력 필드 -> CTA 버튼 -> 외부 계정 로그인 버튼 -> 하단 약관 메시지 순으로 배치
Data type	이메일 주소(String), 외부 계정 토큰(String)
Instruction type	이메일 입력 필드에 “Enter your email to sign up for this app” 플레이스홀더 텍스트 제공
Exit message	● 로그인 성공 시 홈 화면으로 자동 이동 ● 실패 시: “등록되지 않은 이메일이거나 인증에 실패했습니다.”

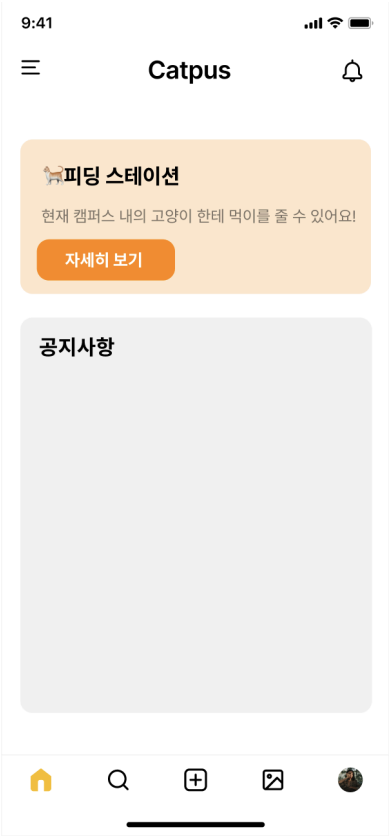


[Figure 2] User Interface - Sign-up

[Table 4-2] User Interface - Sign-up Interface

Name	회원 가입 화면
Purpose/Description	애플리케이션을 처음 사용하는 유저는 회원가입을 거쳐야 한다.
Input source/ Output destination	Client/Server
Unit	Screen
Time/Velocity	-
Relationship with other input/outputs	이름, 이메일, 사용할 비밀번호 / 유효한 이메일인지의 확인 이메일 인증번호 / 인증번호 확인

	닉네임 / 로그인 토큰
Format,Configuration of screen & User Flow	1. 로그인에 사용할 이름, 이메일, 비밀번호를 확인하면 회원가입 버튼이 활성화된다. 2. 사용자가 이메일로 인증 번호를 보낸다. 3. 인증번호로 유효한 이메일인지 확인과정을 거친다. 4. 이메일 인증이 완료되면 닉네임을 입력받아 회원가입을 진행한다. 5. 회원가입에 성공하면 로그인 토큰을 받아 앱의 메인 화면으로 넘어간다.
Format, Configuration of window	모바일 환경 기준, 세로 스크롤 UI, 상단 로고 / 입력 필드 / 버튼 순서로 구성 버튼은 노란색 강조 색상 사용, 모든 입력 필드는 필수 입력값 확인 로직 포함
Data type	이름(String), 이메일 주소(String), 이메일 인증 번호(Integer, 6자리 숫자), 비밀번호(String), 비밀번호 확인(String), 닉네임(String), 활동 목적(String)
Instruction type	각 입력란에서는 플레이스 홀더로 예시 제공 (ex. 이메일 주소, 숫자 6자리, 특수문자 포함 비밀번호 등)
Exit message	● 회원가입 완료 시: “회원가입이 완료되었습니다. 환영합니다!” ● 이메일 인증 실패: “유효하지 않은 인증번호입니다. 다시 시도해주세요.”



[Figure 3] User Interface - Home

[Table 4-3] User Interface - Home Interface

Name	홈 화면
Purpose/Description	앱의 메인 페이지로서 사용자의 주요 활동 현황(고양이 돌봄 상태, My Petstagram 피드, 공지사항 등)을 한눈에 확인할 수 있는 대시보드 역할을 한다.
Input source/ Output destination	Client/Server
Unit	Screen
Time/Velocity	-

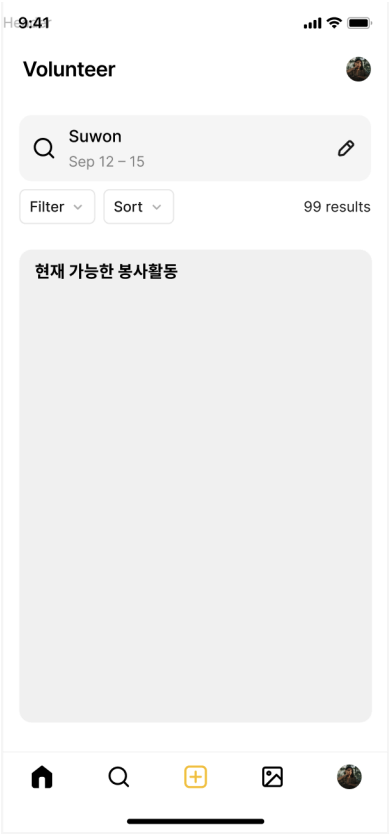
Relationship with other input/outputs	사용자 로그인 세션, 급식소 정보, Petstagram 피드 데이터, 공지사항 데이
Format,Configuration of screen & User Flow	1. 앱 실행 시 로그인 세션 확인 후 홈 화면으로 이동한다. 2. 상단 배너에 앱 로고, 알림 아이콘, 프로필 버튼이 표시된다. 3. 급식소 상태 및 공지사항, 주요 이벤트 등이 카드 형태로 시각화된다. 4. “피딩 참여하기”, “봉사 신청” 등의 빠른 액션 버튼을 통해 즉시 참여 가능하다. 5. 하단 탭 바를 통해 “홈”, ”피드”, ”활동”, ”내 정보” 등의 페이지로 이동할 수 있다.
Format, Configuration of window	● 상단 고정 배너(로고 및 알림), 메인 콘텐츠 영역(상태 카드, 공지사항), 하단 고정 탭 바로 구성
Data type	사용자 ID(String), 피딩 상태 정보(Object), 공지사항 목록(List <String>), 알림 여부(Boolean)
Instruction type	카드 및 버튼에 간결한 안내 문구 삽입 (예: “현재 10개 급식소에 활동 중”)
Exit message	● 데이터 로딩 실패 시: “정보를 불러오는 데 실패했습니다. 다시 시도해주세요.” ● 로그인 세션 만료 시 로그인 화면으로 리디렉션

[Figure 4] User Interface - Cat Feeding Station

[Table 4-4] User Interface - Cat Feeding Station Interface

Name	교내 고양이 급식소 위치 화면
Purpose/Description	캠퍼스 내 길고양이 급식소의 위치 정보를 지도와 함께 시각적으로 제공하여 사용자가 주변 급식소를 쉽게 탐색하고 확인할 수 있도록 하는 화면이다.
Input source/ Output	Client / Server

destination	
Unit	Screen
Time/Velocity	-
Relationship with other input/outputs	사용자 위치 정보, 급식소 위치 데이터(주소, 이름, 좌표), 지도 API
Format,Configuration of screen & User Flow	1. 사용자가 해당 탭에 진입하면 자동으로 현재 캠퍼스 지도가 표시된다. 2. 지도 상에 급식소 위치가 마커로 나타나며, 각 급식소는 리스트 형태로도 함께 제공된다. 3. 주소 정보가 함께 표시되며, 시각적 강조를 해 사용자 접근성을 높인다.
Format, Configuration of window	● 상단 페이지 타이틀, 중앙 지도 뷰(마커 포함), 하단 급식소 리스트(번호, 이름, 주소 포함) 구성 ● 마커는 지도 상위 위치와 매핑되며, 리스트는 카드 스타일 UI로 구성됨
Data type	급식소 ID(String), 이름(String), 주소(String), 위치 좌표(GeoLocation: [위도, 경도])
Instruction type	각 급식소 카드에 “지도에서 보기” 등의 동작 유도 문구 제공 가능
Exit message	● 지도 로딩 실패 시: “지도를 불러오는데 실패했습니다. 네트워크 상태를 확인해주세요.”



[Figure 5] User Interface - Activity Participation

[Table 4-5] User Interface - Activity Participation Interface

Name	활동 참여 화면
Purpose/Description	사용자가 캠퍼스 고양이 돌봄 활동 및 주변 봉사 활동(급식소 관리, 쉘터정리, 캠페인 등)에 참여할 수 있도록 활동 목록을 표시하고 신청할 수 있는 화면이다.
Input source/ Output destination	Client / Server
Unit	Screen
Time/Velocity	-

Relationship with other input/outputs	사용자 인증 상태, 활동 데이터(위치, 일정, 인원 수), 참여자 상태 정보
Format,Configuration of screen & User Flow	1. 사용자가 로그인한 상태로 활동 참여 탭에 접근한다. 2. 화면 상단에서 위치 또는 날짜별 필터링이 가능하다. 3. 각 활동 리스트에는 위치, 날짜, 시간, 필요 인원 등이 표시된다. 4. 사용자는 원하는 활동을 클릭하여 상세 정보를 확인하고 "참여하기" 버튼을 눌러 신청한다. 5. 신청 완료 후 활동 상태(참여 중, 대기 중, 완료 등)가 표시된다.
Format, Configuration of window	● 상단 검색 및 필터 바 -> 활동 목록 카드(필요 인원/상태 등 포함) -> 참여 상태 표기 ● 각 활동은 카드 형태로 구성되며 참여 버튼 포함
Data type	활동 ID(String), 위치(GeoLocation), 일정(DataRange), 참여자 목록([UserId]), 참여 상태(String: '대기 중', '참여 중', '완료')
Instruction type	필터 버튼, 활동 리스트 내 '참여하기' 버튼 등 명확한 작업 유도 UI 제공
Exit message	● 참여 완료 시: "활동 참여가 완료되었습니다." ● 인원 초과 시: "활동 참여 가능한 인원이 마감되었습니다."

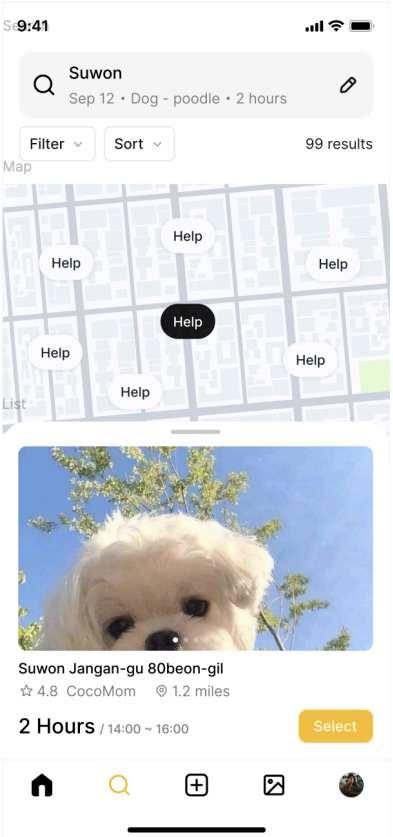
[Figure 6] User Interface - Pet Sitting Request

[Table 4-6] User Interface - Pet Sitting Request Interface

Name	펫시팅 신청 화면
Purpose/Description	반려동물 보호자가 원하는 일정과 조건에 따라 펫시팅 요청을 등록할 수 있는 화면으로, 시터 매칭 기능의 시작점 역할을 한다.
Input source/ Output destination	Client / Server

Unit	Screen
Time/Velocity	-
Relationship with other input/outputs	사용자 인증 상태, 요청 날짜 및 시간, 반려동물 정보, 요청사항 업로드된 이미지
Format,Configuration of screen & User Flow	1. 사용자는 날짜 선택 버튼을 눌러 원하는 일정을 선택하고, 시간 입력에 희망 시간을 입력한다. 2. 반려동물 종류를 텍스트로 입력한다. 3. 반려동물의 사진을 첨부한다. 4. 시터로 고려되었으면 하는 사항과 구체적인 요청사항을 입력한다. 5. 모든 항목이 입력되면 '신청 완료' 버튼을 눌러 서버로 요청이 전송된다. 6. 완료 후 매칭 가능한 시터 목록 화면으로 이동하거나 신청 내역 화면으로 전환된다.
Format, Configuration of window	● 상단 타이틀 '펫시팅 신청', 입력 폼 필드(날짜, 시간, 종류, 사진, 고려사항, 요청사항)와 하단 CTA 버튼으로 구성 ● 모든 필드는 세로 스택 형식으로 정렬
Data type	날짜(Date), 시간(TimeRange: 시작~종료), 반려동물 종류(String), 첨부 이미지(Image File: .jpg, .png), 고려사항(String), 요청사항(String)
Instruction type	● 각 필드에 플레이스홀더 텍스트 제공 (예: "강아지 키워 보신 분이면 좋아요") ● 달력 아이콘 및 파일 선택 아이콘으로 입력을 직관화
Exit message	● 신청 완료 시: "펫시팅 요청이 등록되었습니다. 매칭 가능한 시터를 찾고 있어요." ● 필수 입력 누락 시: "모든 필드를 입력해주세요."

	• 파일 업로드 오류: “사진 업로드에 실패했습니다. 다시 시도해주세요”

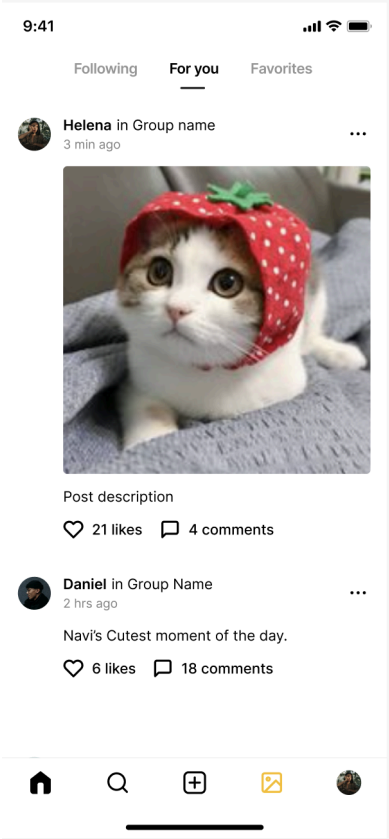


[Figure 7] User Interface - Pet Sitting Matching

[Table 4-7] User Interface - Pet Sitter Matching Interface

Name	펫시터 매칭 화면
Purpose/Description	사용자 간 신뢰 기반의 펫시터 매칭을 위한 화면으로, 요청된 펫시팅에 대해 적합한 시터 목록을 보여주고, 사용자는 시터의 정보를 확인하거나 매칭 요청을 보낼 수 있다.
Input source/ Output destination	Client / Server

Unit	Screen
Time/Velocity	-
Relationship with other input/outputs	사용자 신뢰도, 매칭 요청 데이터, 시터 정보(위치, 배지, 평점), 채팅 시스템
Format,Configuration of screen & User Flow	1. 사용자가 펫시터 요청 화면에서 조건(날짜, 위치 등)을 입력하고 요청을 보낸다. 2. 매칭 가능한 시터들의 목록과 지도 상 위치가 함께 표시된다. 3. 각 시터 카드에는 시터의 프로필 사진, 거리 평점, 이전 기록, 배지 등이 표시된다. 4. 사용자는 시터 상세 프로필 확인 또는 메시지 전송 버튼을 통해 소통할 수 있다.
Format, Configuration of window	● 상단 검색 및 필터 바 -> 지도 뷰 및 결과 수 -> 시터 리스트 영역으로 구성 ● 시터 카드는 프로필 이미지와 주요 정보(거리, 시간, 평점 등)를 요약하여 표시
Data type	사용자 위치(GeoLocation), 시터 정보 객체(이름, 평점:Float, 배지: [String], 거리: Float, 활동 시간: DateRange), 메시지 버튼 상태 (Boolean)
Instruction type	지도에는 Help 마커와 시터 위치 표시, 카드에는 시터 활동 정보 요약 및 “Select” 버튼 제공
Exit message	● 매칭 가능한 시터 없음: “현재 조건에 맞는 시터가 없습니다.”

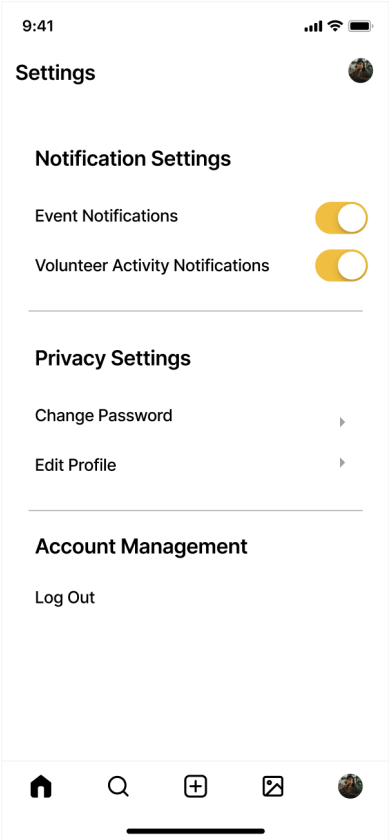


[Figure 8] User Interface - My Petstagram

[Table 4-8] User Interface - My Petstagram Interface

Name	My Petstagram 화면
Purpose/Description	사용자가 반려동물의 사진 및 영상을 게시하고, 다른 사용자들과 댓글, 좋아요, 공유 기능 등을 통해 소통할 수 있는 커뮤니티 피드 화면이다.
Input source/ Output destination	Client / Server
Unit	Screen
Time/Velocity	-
Relationship with other	사용자 ID, 게시물(사진/영상), 좋아요/댓글/공유 데이터, 태그 및

input/outputs	해시태그 정보
Format,Configuration of screen & User Flow	1. 사용자가 상단 탭에서 피드 범위를 선택할 수 있다. (예: Following, For you, Favorites). 2. 피드에는 다른 사용자의 게시물이 순차적으로 표시되며, 각 게시물에는 이미지/영상, 설명, 좋아요 수, 댓글 수가 함께 제공된다. 3. 게시자 프로필 클릭 시 해당 사용자의 상세 프로필로 이동한다. 4. 하단 탭 바에서 다른 화면으로 이동할 수 있다.
Format, Configuration of window	● 상단 피드 필터 탭 -> 피드 카드 리스트 -> 하단 탭 바 UI 구성 ● 각 카드에는 프로필 이미지, 게시물 설명, 해시태그, 아이콘 기반의 좋아요/댓글/공유 버튼이 포함
Data type	게시물 ID(String), 이미지/영상 URL(String), 설명(String), 사용자 ID(String), 댓글 수(Integer), 좋아요 수(Integer), 태그/해시태그 ([String])
Instruction type	포스트 작성 시 설명 및 해시태그 입력란 제공, 각 버튼에는 직관적인 아이콘 사용
Exit message	● 게시물 로딩 실패: “피드를 불러올 수 없습니다. 다시 시도해주세요.”



[Figure 9] User Interface - Settings

[Table 4-9] User Interface - Settings Interface

Name	설정 화면
Purpose/Description	사용자가 계정 관련 설정, 알림 수신 여부, 개인정보 보호, 비밀번호 변경, 로그아웃 등을 직접 제어할 수 있는 설정 화면이다.
Input source/ Output destination	Client / Server
Unit	Screen
Time/Velocity	-
Relationship with other	사용자 인증 상태, 푸시 알림 설정값, 프로필 정보, 계정 로그아웃 요청

input/outputs	등
Format,Configuration of screen & User Flow	1. 사용자가 하단 탭 바의 '내 정보' 또는 상단 메뉴 버튼을 통해 설정 화면에 진입한다. 2. 알림 설정(예: 이벤트, 봉사활동 알림)을 토글 버튼으로 ON/OFF할 수 있다. 3. 개인정보 설정에서 비밀번호 변경 및 프로필 편집이 가능하다. 4. 계정 관리 섹션에서 로그아웃을 수행할 수 있다. 5. 모든 변경 사항은 실시간 저장되어가 저장 버튼 클릭 시 서버로 전송된다.
Format, Configuration of window	● 화면은 크게 Notification Settings, Privacy Settings, Account Management의 섹션으로 나뉘며, 각 항목은 리스트 형태로 구성됨 ● 알림 설정은 토글 스위치로 구성되며 나머지 항목은 클릭 시 상세 설정 화면으로 이동
Data type	사용자 ID(String), 알림 설정(Boolean), 비밀번호(String), 프로필 정보(Object), 로그인 상태(Boolean)
Instruction type	각 설정 항목에 명확한 제목과 부가 설명 텍스트 또는 아이콘 제공
Exit message	● 설정 저장 성공: "설정이 저장되었습니다." ● 로그아웃 시: "로그아웃 되었습니다."

3.1.1.1 화면 배치 제약 사항

- **반응형 디자인:**
  - 다양한 화면 크기 (스마트폰, 태블릿)에서 일관된 사용자 경험 제공
  - 주요 구성 요소는 화면 크기에 따라 자동으로 조정
- **일관된 스타일:**
  - 앱 전반에 걸쳐 통일된 색상 팔레트, 글꼴 크기, 버튼 스타일 사용

    - 가이드라인에 따라 Campus Cat의 색상 테마 적용
- **탭 및 네비게이션:**
    - 하단 고정 탭 바를 사용하여 주요 페이지 간의 빠른 전환 지원
    - 상단에는 페이지 제목과 검색, 설정 버튼 배치

3.1.1.2 표준 버튼 및 기능

- **기본 버튼 스타일:**
    - 주요 작업 (참여, 업로드, 수정) 버튼은 강한 색상으로 강조
    - 취소, 돌아가기, 삭제와 같은 부정 작업은 중립적인 색상 사용
- **아이콘 및 단축키:**
    - 자주 사용하는 기능 (예: 좋아요, 댓글, 공유)에 대한 직관적인 아이콘 사용
    - 화면 전환 시 애니메이션 효과 추가
- **알림 및 메시지:**
    - 새로운 활동, 펫 시터 매칭, 댓글 알림은 실시간 푸시 메시지로 전송
    - 중요한 시스템 메시지는 상단 배너 또는 팝업으로 표시

3.1.1.3 오류 메시지 표준

- **사용자 입력 오류:**
    - 필수 입력란 미기입 시 "이 필드를 입력해주세요."
    - 잘못된 형식 입력 시 "유효하지 않은 형식입니다. 다시 시도해주세요."
- **네트워크 오류:**
    - "네트워크 연결이 불안정합니다. 다시 시도해주세요."
- **시스템 오류:**
    - "알 수 없는 오류가 발생했습니다. 고객 지원팀에 문의해주세요."

3.1.2 Hardware Interfaces

3.1.2.1 지원 장치 유형

앱은 다양한 모바일 장치에서의 호환성과 최적의 성능을 보장하기 위해 다음과 같은 장치 유형을 지원합니다:

- **스마트폰 및 태블릿:**
  - iOS(버전 15 이상), Android(버전 12 이상) 운영체제에서 지원하며 동작합니다.
  - 최소 하드웨어 사양:

[Table 5] Table of Minimum Hardware Specifications

CPU	듀얼 코어 1.5GHz 이상
RAM	3 GB 이상
Storage	최소 100MB (캐시 데이터 및 이미지 제외)
Network	Wi-Fi 802.11 a/b/g/n/ac, 4G LTE, 5G 또는 그 이상

3.1.2.2 데이터 및 제어 상호작용

앱은 다양한 하드웨어 구성 요소를 활용하여 데이터를 수집하고, 사용자 입력을 처리하며, 빠르고 반응성 높은 사용자 경험을 제공합니다. 다음은 주요 하드웨어와의 상호 작용 방식입니다:

- **Camera:**
  - Petstagram의 사진 및 비디오 업로드.
  - 고해상도 이미지 캡처를 위한 기기의 기본 카메라 API 사용.
  - 실시간 이미지 처리 및 압축을 지원하여 빠른 업로드 가능.
- **Location Services:**

  - 피딩 스테이션 탐색 및 이벤트 위치 표시.
  - GPS와 Wi-Fi 위치 정보를 사용하여 주변 피딩 스테이션 및 이벤트 위치 탐색.
  - 실시간 위치 업데이트를 통한 펫 시터 매칭 및 봉사 활동 관리.
  - 지오펜싱 (Geofencing)을 활용하여 사용자의 위치에 따른 맞춤형 알림 전송
- **저장소 및 파일 시스템:**
  - 사용자 데이터, 이미지, 오프라인 접근을 위한 로컬 캐싱.
  - 임시 데이터와 캐시 저장을 위한 로컬 스토리지.
  - 민감한 데이터 (예: 사용자 인증 정보, 펫 프로필)의 안전한 저장
  - 확장성을 위해 AWS S3와 같은 클라우드 스토리지와 통합

3.1.2.3 통신 프로토콜

앱은 하드웨어 구성 요소와의 효율적인 데이터 교환을 위해 다양한 산업 표준 프로토콜을 사용합니다:

- **Wi-Fi 및 Cellular:**
  - 실시간 알림, 사진 업로드, API 통신을 위한 고속 데이터 전송
  - Wi-Fi와 셀룰러 네트워크 간의 자동 전환을 통한 끊김 없는 연결, 데이터 전송의 안정성을 위해 네트워크 상태에 따른 적응형 전송 속도 관리.

3.1.2.4 전력 관리 및 최적화

사용자 경험 향상을위해 모바일 애플리케이션의 효율적인 전력 관리를 구현합니다:

- **Battery Management:**
  - 배경 작업 최소화를 통한 전력 소비 감소.
  - 배터리 수준에 따른 적응형 데이터 동기화, 중요한 이벤트가 아닌 경우 배경 데이터 처리 제한.
- **Resource Optimization:**
  - 저사양 기기에서도 부드러운 앱 성능 보장

- CPU와 메모리의 효율적인 사용, 경량화된 백그라운드 프로세스를 통해 발열 및 전력 소모 감소

3.1.2.5 보안 및 개인정보 보호

앱은 사용자 데이터를 보호하고 하드웨어 구성 요소와의 안전한 상호 작용을 보장하기 위해 다음과 같은 보안 조치를 구현합니다:

- **권한 관리:**
  - 카메라, 위치, 마이크 등의 하드웨어 접근에 대한 엄격한 권한 관리.
  - 데이터 사용 정책에 대한 명확한 안내.
- **데이터 암호화:**
  - 모든 저장된 데이터의 기기 수준 암호화.
  - TLS 1.3을 사용한 클라이언트-서버 간 데이터 전송 암호화.
  - 민감한 데이터의 안전한 저장을 위한 AES-256 암호화 적용.

3.1.3 Software Interfaces

3.1.3.1 백엔드 서버

- **Node.js (버전 18 이상)**
- 역할: 사용자 인증, 데이터 관리, 실시간 알림 처리
- RESTful API와 WebSocket을 통한 클라이언트와의 통신
- 데이터 유효성 검사, 세션 관리, 인증 처리 (JWT 사용)

1) 데이터베이스

- **MongoDB (버전 6.0 이상)**
- 역할: 사용자 프로필, 활동 로그, 펫 정보, 피드백 데이터 저장

- 데이터 모델링: 컬렉션 구조 (Users, Pets, Activities, Reviews)
- 데이터 일관성을 위한 트랜잭션 지원 및 인덱스 최적화
- 자동 백업 및 복구 기능

2) 외부 API

- **지도 및 위치 서비스:** Google Maps API 또는 OpenStreetMap
    - 기능: 피딩 스테이션 위치 표시, 거리 계산, 경로 안내
- **결제 게이트웨이:** Stripe, PayPal
    - 기능: 기부 결제, 펫 케어 서비스 결제 처리
- **푸시 알림:** Firebase Cloud Messaging (FCM), Apple Push Notification Service (APNs)
    - 기능: 실시간 알림, 이벤트 공지

3) 파일 스토리지

- **AWS S3 또는 유사한 클라우드 스토리지**
- 역할: 사진 및 비디오 파일 저장 (e.g., My Petstagram)
- 이미지 최적화, 압축, CDN을 통한 빠른 로딩 속도

4) 보안 서비스

- **JWT (JSON Web Token)**
- 역할: 사용자의 인증 및 권한 관리
- 데이터 암호화 및 서명 기능으로 보안 강화

3.1.3.2 데이터 흐름 및 메시지 형식

- **데이터 입력 (Incoming Data):**
    - 사용자 인증 요청 (로그인, 회원가입)

    - 펫 케어 신청, 피딩 스테이션 참여
    - Petstagram 사진 업로드
    - 피드백 및 리뷰 제출
- **데이터 출력 (Outgoing Data):**
    - 사용자 프로필 및 활동 로그
    - 펫 정보 및 레벨 상태
    - 실시간 알림 메시지
    - 피드 데이터 (My Petstagram 포스트)
    - 매칭된 펫 시터 정보
- **메시지 형식:**
    - 주로 **JSON (JavaScript Object Notation)** 사용
    - 경량화된 구조로 모바일 클라이언트와의 빠른 데이터 교환 가능
    - 이미지나 파일 업로드의 경우 MMIE 형식 사용

3.1.3.3 서비스와 통신 방식

- **RESTful API:**
    - 표준 HTTP 메서드 (GET, POST, PUT, DELETE) 사용
    - 데이터 일관성을 유지하기 위한 트랜잭션 처리
    - JWT를 통한 인증 및 접근 제어
- **실시간 통신:**
    - WebSocket을 통한 양방향 실시간 데이터 교환
    - 실시간 피드백, 펫 시터 매칭, 알림 전송
    - 메시지 큐 (RabbitMQ, Redis Streams) 사용 가능
- **파일 전송:**
    - 대용량 미디어 파일의 경우 AWS S3와 같은 외부 스토리지로 직접 업로드
    - 데이터 압축 및 최적화로 전송 속도 개선

3.1.3.4 데이터 공유 메커니즘

- **전역 데이터 영역 (Global Data Area):**
  - 멀티태스킹 환경에서의 데이터 일관성 유지
  - 공유 데이터에 대한 잠금 메커니즘 (Locking) 사용
- **멀티플랫폼 지원:**
  - 모바일, 웹, 데스크탑 클라이언트에서 동일한 데이터 접근 가능
  - 데이터 일관성을 위한 중앙 집중식 서버 구조
- **API 문서:**
  - OpenAPI (Swagger) 문서로 각 엔드포인트의 데이터 형식, 응답 구조, 오류 코드 정의
  - 유지보수를 위한 명확한 인터페이스 명세서 포함

3.1.4 Communications Interfaces

3.1.4.1 Email

- 사용자 계정 활성화, 비밀번호 재설정, 중요 알림 전송을 위해 이메일 기능이 필요합니다.
- API 기반 이메일 서비스 (예: SendGrid, Amazon SES) 사용 가능
- 주요 기능:
  - 회원가입 확인 이메일
  - 비밀번호 재설정 링크 전송
  - 봉사 활동 알림 및 이벤트 초대

3.1.4.2 Web Browser

- 앱의 웹 버전 또는 웹 포털이 존재하는 경우 주요 웹 브라우저 (Chrome, Safari, Firefox, Edge)와의 호환성 필요
- 웹 페이지는 HTTPS 프로토콜을 사용하여 안전하게 통신
- 웹 소켓 (WebSocket) 및 AJAX를 통한 실시간 데이터 전송 지원

- HTML5, CSS3, JavaScript 기반의 동적 페이지 로딩 및 사용자 입력 처리

3.1.4.3 Network Server Protocol

- **HTTPS (Hypertext Transfer Protocol Secure)**
  - 기본적인 데이터 통신 프로토콜로, 클라이언트와 서버 간의 모든 통신을 암호화하여 전송
  - TLS 1.3을 사용하여 데이터 기밀성 및 무결성 보장
  - 인증서를 통한 서버 신뢰성 확보
- **WebSocket**
  - 양방향 실시간 통신을 위해 사용
  - 펫 시터 매칭, 피드 업데이트, 실시간 채팅 등의 기능에 적합
  - 낮은 지연 시간과 효율적인 데이터 전송 가능
- **FTP (File Transfer Protocol, 선택 사항)**
  - 이미지나 동영상 파일의 대량 전송 시 사용 가능
  - 보안을 위해 SFTP (Secure FTP) 또는 FTPS (FTP Secure) 권장

3.1.4.4 Communication Standard

- **TLS (Transport Layer Security) 1.3**
  - 모든 클라이언트-서버 통신의 암호화 보장
  - 데이터 무결성, 기밀성, 인증을 제공
- **WebSocket**
  - 실시간 통신을 위한 저지연, 양방향 프로토콜
  - HTTP 업그레이드를 통해 초기 연결 설정

3.1.4.5 Message Format

- **JSON (JavaScript Object Notation)**
  - 대부분의 데이터 전송에 사용
  - 경량화된 구조로 모바일 기기에서의 데이터 처리 효율성 향상

- 예시:

```
{
  "user_id": "12345",
  "activity": "Pet Sitting",
  "status": "Confirmed",
  "timestamp": "2025-05-07T12:34:56Z"
}
```

- **XML (Extensible Markup Language, 선택 사항)**
  - 외부 서비스와의 통합이 필요한 경우 사용 가능
  - 보다 복잡한 데이터 구조를 표현하는 데 적합
- **MIME (Multipurpose Internet Mail Extensions)**
  - 파일 업로드 시 적절한 파일 형식 지정 (e.g., image/jpeg, application/json)

3.1.4.6 Security and Encryption Issues

- **데이터 암호화:**
  - 모든 민감한 데이터는 전송 중 TLS 1.3으로 암호화
  - 서버 측에서 저장되는 데이터는 AES-256으로 암호화
- **인증:**
  - JWT (JSON Web Token) 기반의 인증 시스템
  - 세션 탈취 방지를 위한 만료 시간 설정 및 재발급 기능
- **데이터 무결성:**
  - HMAC (Hash-based Message Authentication Code)로 데이터의 무결성 보장
- **접근 제어:**
  - RBAC (Role-Based Access Control) 또는 ABAC (Attribute-Based Access Control) 사용

3.1.4.7 Data Transfer Speed and Synchronization Mechanism

- **데이터 전송 속도 최적화:**
  - gzip, Brotli와 같은 데이터 압축 알고리즘 사용
  - 이미지와 동영상은 미리보기 이미지 생성으로 전송 시간 단축
- **동기화:**
  - 실시간으로 사용자 상태, 펫 케어 활동, 알림 상태 동기화
  - 데이터 충돌 방지를 위한 버전 관리 및 병합 전략
- **오프라인 모드:**
  - 네트워크 연결이 불안정한 환경에서도 데이터 손실 없이 작업 가능하도록 로컬 캐시 사용

3.2 Functional Requirements

3.2.1 User

[Table 6-1] Use case of Sign-In

Use case	회원가입
Actor	신규 사용자
Description	회원가입은 새로운 사용자가 계정을 생성하고 시스템에 등록하기 위해 필요한 정보를 입력하는 절차이다.
Normal Courses	신규 사용자가 시스템을 사용하고자 함 사용자는 로그인 화면에서 회원가입 옵션을 선택함 사용자는 이름, 이메일, 비밀번호 등 필수 정보(TBD)를 입력함 시스템은 입력 값을 검증하고 기존 계정 존재 여부를 확인함 모든 검증을 통과하면 시스템은 새 계정을 생성하고 로그인 페이지로 이동하거나 입력된 정보를 바탕으로 자동 로그인함

Pre-condition	사용자는 아직 시스템에 등록되어 있지 않은 상태임
Post-condition	시스템에 새로운 계정이 생성됨
Assumptions	N/A

[Table 6-2] Use case of Profile Settings

Use case	**프로필 설정**
Actor	회원가입을 마친 사용자
Description	회원가입 이후 사용자는 프로필 이미지, 닉네임, 관심있는 동물, 간단한 자기소개 문구 등을 설정할 수 있으며, 이 정보는 다른 사용자에게 노출됨
Normal Courses	사용자가 로그인한 상태에서 프로필 설정 화면으로 이동함 사용자는 프로필 이미지, 닉네임, 관심 동물, 자기소개 항목을 입력하거나 수정함 사용자가 입력을 마치고 저장 버튼을 클릭함 시스템은 입력된 정보를 검증하고, 검증에 성공 시 해당 프로필 정보를 저장함 업데이트된 프로필 정보는 사용자 프로필과 다른 서비스(펫시터 매칭, 봉사활동 등) 화면에 노출됨
Pre-condition	사용자는 시스템에 로그인된 상태여야 함
Post-condition	프로필 정보가 시스템에 저장되고, 관련 기능에서 활용 됨
Assumptions	프로필 입력 값들은 필수 항목이 아니며, 일부 항목은 비워둘 수 있음

3.2.2 Auth

[Table 6-3] Use case of Login/Logout

Use case	**로그인/로그아웃**
Actor	회원가입을 마친 사용자

Description	로그인은 등록된 사용자가 시스템에 접근하기 위해 ID와 비밀번호를 입력하여 인증 받는 절차를 말함. 로그아웃은 로그인된 사용자가 시스템에서 나가는 절차를 말함.
Normal Courses	<로그인> 등록된 사용자가 시스템의 서비스를 이용하고자 함 사용자는 회원가입 시 입력한 ID 와 비밀번호를 입력함 입력한 정보가 정확하면 시스템에 로그인되어 서비스를 이용할 수 있게 됨 <로그아웃> 사용자가 시스템에서 나가고자 할 경우, 프로필 페이지(TBD)의 로그아웃 버튼을 클릭함. 다음에 어플리케이션을 실행할 경우, 사용자는 다시 로그인 해야 함. 로그아웃 없이 어플리케이션을 종료할 경우, 시스템은 세션 정보를 자동으로 종료함.
Pre-condition	<로그인> 사용자는 이미 회원가입 절차를 마친 상태여야 함 <로그아웃> 사용자는 시스템에 로그인한 상태여야 함
Post-condition	사용자는 네트워크에 연결됨
Assumptions	N/A

[Table 6-4] Use case of Access Token Refresh

Use case	**로그인 갱신**
Actor	로그인한 사용자 및 시스템
Description	로그인 갱신이란 만료된 JWT access_token을 새로운 access_token으로 재발급 받는 과정을 의미함
Normal Courses	사용자가 만료된 access_token을 헤더에 포함한 요청을 시스템에 전송함 시스템은 토큰이 만료되었다고 사용자에게 응답함

	사용자는 refresh_token을 시스템에 보냄 시스템은 refresh_token 정보를 저장하고 있던 정보와 비교하여 검증함 검증에 성공 시 사용자에게 새로운 access_token을 발급함
Pre-condition	사용자는 만료된 access_token을 보유한 상태여야 함
Post-condition	사용자는 만료되지 않은 새로운 access_token을 발급 받음
Assumptions	시스템은 refresh_token 정보를 저장하고 있음

3.2.3 Cat Care and Volunteer Activities

[Table 6-5] Use case of View Feeding Stations

Use case	**고양이 급식소 위치 확인**
Actor	회원가입을 마친 사용자
Description	사용자는 지도 기반 인터페이스를 통해 캠퍼스 내에 설치된 고양이 급식소의 위치를 시각적으로 확인할 수 있음
Normal Courses	사용자가 로그인한 상태로 어플리케이션에 접속함 사용자는 메뉴에서 “급식소 위치 보기”(TBD)를 선택함 시스템은 지도에 캠퍼스 내 급식소 위치를 마커로 표시함 사용자가 마커를 클릭하면 세부 정보(급식소 이름, 위치 설명 등)가 표시됨
Pre-condition	사용자는 시스템에 로그인된 상태여야 함
Post-condition	사용자는 원하는 급식소 위치 정보를 확인할 수 있음
Assumptions	급식소의 위치 데이터는 사전에 등록되어 있어야 함

[Table 6-6] Use case of Join Volunteering

Use case	**봉사활동 참여**

Actor	회원가입을 마친 사용자
Description	사용자는 교내 급식소 관리, 물품 정리 등 고양이 돌봄 활동과 외부 보호소의 자율 봉사 활동에 신청하고, 참여 시 활동 내역이 기록됨
Normal Courses	사용자가 봉사 활동 참여 메뉴에 접근함 시스템은 참여 가능한 봉사 활동 목록(시간, 장소 등)을 표시함 사용자는 원하는 활동을 선택하고 참여 신청 함 시스템은 신청 결과를 확인하고 사용자에게 알림 사용자가 실제로 봉사 활동에 참여하면 관리자는 활동 완료 상태로 기록하거나 사용자가 인증 정보를 제출함 시스템은 해당 활동을 사용자 활동 이력에 저장함
Pre-condition	사용자는 시스템에 로그인된 상태여야 함
Post-condition	봉사 활동 참여 기록이 사용자 이력에 반영됨
Assumptions	일부 봉사 활동의 경우 승인 절차가 존재하므로, 활동 완료 여부를 확인하는 절차는 수동으로 진행 됨

3.2.4 Pet Sitting Matching System

[Table 6-7] Use case of Request Pet Sitting

Use case	**펫시터(Pet Sitter) 요청 등록**
Actor	반려동물의 보호자
Description	보호자는 날짜, 시간, 요청사항(TBD) 등을 입력하여 펫시터 요청을 등록할 수 있음
Normal Courses	사용자가 로그인한 상태에서 펫시터 요청 메뉴로 이동함 사용자는 날짜, 시간, 반려동물 종류, 요청사항 등을 입력함 시스템은 입력값을 검증하고 펫시터 요청을 저장함 요청은 적합한 펫시터 목록에 노출됨

Pre-condition	사용자는 시스템에 로그인된 상태여야 함
Post-condition	요청이 등록되어 펫시터들에게 공개됨
Assumptions	기본적인 반려동물 정보가 등록되어 있어야 함

[Table 6-8] Use case of Accept/Decline Request

Use case	**펫시터(Pet Sitter) 요청 수락 또는 거절**
Actor	펫시터 사용자
Description	펫시터는 본인의 일정과 조건에 따라 보호자의 요청을 수락하거나 거절할 수 있으며, 매칭이 성사되면 채팅 기능이 활성화됨
Normal Courses	펫시터가 로그인한 상태로 펫시터 요청 목록을 확인함 사용자는 목록에서 요청 상세 정보를 확인함 요청 조건이 맞을 경우, 수락 버튼을 눌러 매칭을 성사시킴 조건이 맞지 않으면 거절을 선택할 수 있음 매칭이 성사되면 시스템은 채팅 기능을 활성화 하고 알림을 전송함
Pre-condition	펫시터는 로그인된 상태이며, 펫시터 활동 기준을 충족해야 함
Post-condition	요청이 수락되거나 거절되며, 매칭 여부에 따라 후속 기능(채팅)이 활성화 됨
Assumptions	동일한 펫시터 요청에 대해 다수의 펫시터가 접근 가능하며, 최초 수락한 펫시터 사용자에게 우선권이 주어짐

[Table 6-9] Use case of View Matches

Use case	**매칭 내역 확인**
Actor	펫시터 사용자, 반려동물의 보호자
Description	과거의 펫시팅 매칭 내역은 사용자 프로필에 저장되며, 후기 작성 및 활동 점수

	집계에 활용됨
Normal Courses	사용자는 프로필 페이지로 이동함 프로필에는 과거에 수락한 또는 요청한 펫시팅 내역이 표시됨 각 항목에는 활동일자, 펫시터/보호자 여부, 후기 작성 여부(TBD) 등이 함께 표시됨 후기를 작성하지 않은 내역은 “후기 작성” 버튼(TBD)을 통해 후기 작성 화면으로 이동할 수 있음
Pre-condition	사용자는 로그인된 상태여야함
Post-condition	펫시팅 활동 기록이 사용자 프로필에 반영됨
Assumptions	모든 보호자/펫시터 매칭 기록은 데이터베이스에 저장되어야 하며, 수정 불가능한 상태로 유지됨

3.2.5 Review and Trust Level System

[Table 6-10] Use case of Write Review

Use case	**리뷰 작성**
Actor	매칭 완료된 사용자(반려동물의 보호자)
Description	매칭 완료 후, 사용자는 상대방에 대해 별점과 간단한 피드백을 포함한 리뷰를 작성할 수 있음
Normal Courses	매칭 완료 후 시스템은 사용자(보호자)에게 리뷰 작성 알림을 전송함 사용자는 리뷰 작성 화면에서 별점과 간단한 피드백을 입력함 시스템은 입력된 내용을 저장함 작성된 리뷰는 상대방(펫시터) 프로필에 반영 됨
Pre-condition	매칭이 완료된 상태여야 하며, 해당 매칭에 대해 리뷰를 작성하지 않은 상태여야 함

Post-condition	리뷰가 등록되어 사용자(펫시터)의 신뢰도 평가에 반영됨
Assumptions	각 매칭에 대해 한 번만 리뷰 작성이 가능하며, 작성된 리뷰는 수정할 수 없음

[Table 6-11] Use case of Trust Level Calculation

Use case	**신뢰도 레벨 계산**
Actor	시스템
Description	시스템은 사용자(반려동물 보호자) 리뷰, 누적 활동 수, 신고 여부 등의 데이터를 바탕으로 신뢰도 레벨을 자동으로 계산함
Normal Courses	시스템은 주기적으로, 리뷰 등록 시마다 해당 사용자의 데이터를 수집함 리뷰 평점 평균, 누적 봉사/펫시팅 횟수, 신고 횟수 등을 종합적으로 분석함 분석된 결과를 기반으로 사용자의 신뢰도 레벨을 수치 또는 등급(TBD)으로 산정함 계산된 신뢰도 레벨은 사용자 프로필에 실시간 반영됨
Pre-condition	사용자에게 적어도 하나 이상의 활동 기록 또는 리뷰 데이터가 있어야 함
Post-condition	신뢰도 수치가 갱신되어 사용자 프로필 정보에 반영됨
Assumptions	신뢰도 레벨 계산 알고리즘은 사전 정의된 가중치 기준에 따라 동작함

3.2.6 Badge and Reward System

[Table 6-12] Use case of Earn Badge

Use case	**뱃지 획득**
Actor	회원가입을 완료한 사용자
Description	사용자는 시스템에서 정해진 기준(TBD)을 달성하면 특정 뱃지를 자동으로 획득함

Normal Courses	시스템은 사용자의 활동 데이터를 주기적으로 분석함 사용자가 특정 조건을 만족할 경우, 시스템은 해당 조건에 대응하는 뱃지를 부여함 부여된 뱃지는 사용자 프로필에 반영되며 해당 내용을 사용자에게 알림으로 전송함
Pre-condition	사용자는 활동 조건을 만족하는 기록을 가지고 있어야 함
Post-condition	뱃지가 사용자 프로필에 등록됨
Assumptions	뱃지 부여 조건이 명확하게 사전 정의되어 있음

[Table 6-13] Use case of View Badges

Use case	**내 뱃지 보기**
Actor	회원가입을 완료한 사용자
Description	사용자는 자신이 획득한 뱃지를 프로필에서 확인할 수 있으며, 해당 프로필 정보는 다른 사용자게엑도 공개되어 신뢰도 지표로 활용됨
Normal Courses	사용자가 프로필 페이지에 접속함 시스템은 사용자에게 부여된 모든 뱃지 목록을 출력함 다른 사용자가 해당 사용자 프로필을 열람할 경우, 뱃지 목록도 같이 표시함 각 뱃지는 간단한 설명(TBD) 및 획득 조건(TBD)을 함께 표시함
Pre-condition	사용자는 로그인한 상태여야 함
Post-condition	뱃지 정보가 사용자 자신 및 다른 사용자에게 표시됨
Assumptions	N/A

3.2.7 SNS Sharing and Community

[Table 6-14] Use case of Cat Community

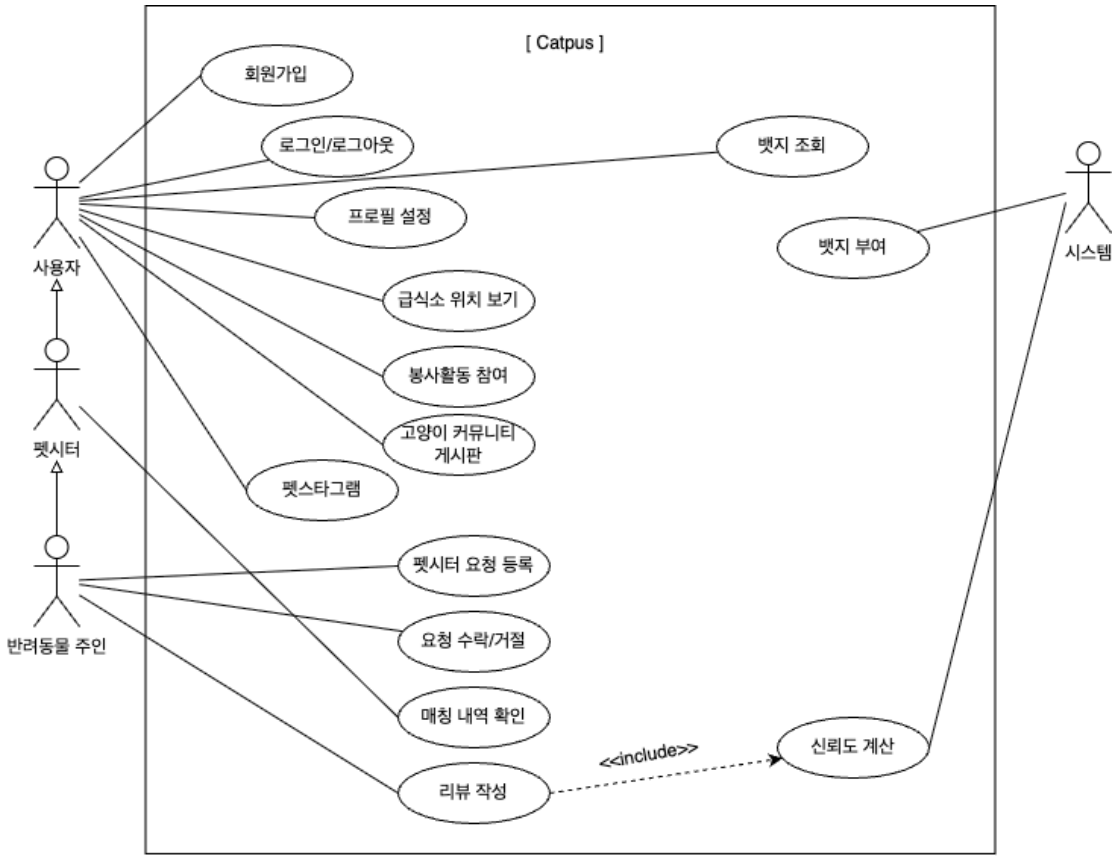
Use case	고양이별 커뮤니티 게시판 이용
Actor	로그인한 사용자
Description	교내에 출현하는 고양이마다 별도로 구성된 게시판이 존재하고, 사용자는 해당 게시판에서 고양이의 사진, 위치, 건강 상태, 최근 목격 정보 등을 등록하고 다른 사용자가 올린 정보를 열람할 수 있음
Normal Courses	사용자가 커뮤니티 메뉴에서 특정 고양이를 선택함 해당 고양이에 대한 게시판으로 이동하고, 게시글을 열람할 수 있음 사용자는 게시글 작성 버튼을 눌러 게시글 등록 화면으로 이동함 등록 화면에서 사진, 목격 정보, 상태 등을 입력함 시스템은 입력 내용을 검증하고 게시글을 게시판에 등록함 다른 사용자도 해당 게시글에 댓글이나 좋아요를 남길 수 있음
Pre-condition	사용자는 로그인한 상태여야 하며, 게시판에 접근 권한이 있어야 함
Post-condition	게시판에 게시글이 등록되고, 다른 사용자가 열람할 수 있음
Assumptions	고양이별 게시판이 사전에 생성되어 있어야 함

[Table 6-15] Use case of Petstagram

Use case	펫스타그램 이용
Actor	로그인한 사용자
Description	사용자는 자신의 반려동물 사진, 일상 등을 펫스타그램에 업로드하고, 다른 사용자들과 댓글, 좋아요 등으로 소통할 수 있음

Normal Courses	사용자가 메뉴에서 “펫스타그램” 항목으로 이동함 사용자는 새 게시글 작성 버튼을 클릭하여 이미지 및 설명을 입력함 시스템은 입력값을 검증한 후 게시글을 등록함 다른 사용자는 해당 게시글에 댓글을 남기거나 좋아요를 누를 수 있음 작성자는 댓글에 대댓글을 남기거나 게시글을 삭제할 수 있음
Pre-condition	사용자는 로그인한 상태여야 함
Post-condition	게시글이 펫스타그램 게시판에 등록되고, 다른 사용자가 열람할 수 있음
Assumptions	N/A

3.2.8 Use Case Diagrams



[Figure 10] Use Case Diagrams

3.3 Nonfunctional Requirements

3.3.1 Performance Requirements

- **Response Time**: 정상적인 네트워크 환경에서 사용자 요청에 대해 2초 이내로 응답해야 한다. 검색 쿼리와 지도 로딩은 3초 이내에 완료되어야 한다.
- **Load Capacity**: 시스템은 최소 5,000명의 동시 사용자를 성능 저하 없이 지원해야 한다.
- **Battery Usage**: 배터리 소모를 최적화하여 꼭 필요한 기능만 백그라운드에서 실행되도록 해야 한다. 또한 앱 사용 중 시간당 배터리 소모량은 15% 이하로 유지되어야 한다.
- **Scalability**: 사용자 수가 50% 증가하더라도 정상적으로 운영될 수 있어야 한다.
- **Offline Functionality**: 이전에 블러온 급식소 위치나 저장된 반려동물 및 캠퍼스 고양이 정보와 같은 핵심 기능은 인터넷 연결 없이도 이용할 수 있어야 한다.
- **Map Performance**: 사용자가 지도 화면을 열었을 때 3초 이내에 캠퍼스 내 모든 급식소가 표시되어야 한다.
- **Media Loading**: Petstagram 피드의 이미지는 단계적으로 로드되어야 하며, jpg/png 파일 형식 대신 webp 파일 형식으로 이미지를 저장해 이미지 최적화를 진행한다.

3.3.2 Safety Requirements

- **Pet Health Protocols**: 펫시팅이나 봉사 활동에 참여하는 사용자에게 기본적인 반려동물 건강 및 안전 수칙을 제공해야 한다.
- **Emergency Contact System**: 동물 건강에 위급 상황이 발생했을 때 즉시 대응할 수 있도록 비상 연락처 정보를 관리해야한다.
- **Activity Verification:** 실제로 급식과 돌봄 활동이 이루어졌음을 확인하기 위해 사진 증거 방법이 필요하다
- **Health Alert System**: 커뮤니티 게시판에는 캠퍼스 고양이의 건강 문제를 알릴 수 있는 신고 기능이 있어야 하며, 이는 활동 중인 봉사자들에게 알림을 보내야 한다.

- **First-Time User Guidance**: 처음 사용하는 회원은 동물 돌봄 활동에 참여하기 전에 필수 안전 지침을 반드시 확인해야 한다.

3.3.3 Security Requirements

- **Data Encryption**: 모든 사용자 데이터는 전송 중(TLS 1.3사용) 과 저장 시(AES-256) 모두 암호화되어야 한다.
- **Authentication**: 이메일 인증과 카카오 인증을 포함한 안전한 인증 방식을 갖추어야 한다.
- **Privacy Protection**: 사용자의 위치 정보는 위치 기반 서비스를 이용할 때만 수집되어야 하며, 정해진 기간 동안만 보관되어야 한다.
- **Data Retention**: 사용자 데이터는 필요한 기간 동안만 보존하고 계정이 삭제될 경우 영구적으로 데이터베이스에서 제거되어야 한다.
- **Secure Communications**: 앱 내 사용자 간 모든 대화는 종단간 암호화되어야 한다.
- **Vulnerability Management**: 정기적인 보안 평가를 실시하고, 보안 패치는 발표 후 7일 이내에 적용해야 한다.
- **Image Privacy**: 의도치 않은 위치 공개나 개인정보 유출을 방지하기 위해 업로드된 이미지의 메타 데이터를 제거해야 한다.

3.3.4 Software Quality Attributes

- **Reliability**: 이용자가 많은 시간대(오전 10시 ~ 오후 10시)에 99.5% 이상의 서비스 가용성을 유지해야 한다.
- **Usability**: UI는 처음 이용자의 85% 이상이 도움 없이도 주요 기능을 사용할 수 있을 만큼 직관적이어야 한다.
- **Maintainability**: 코드는 향후 업데이트를 쉽게 할 수 있도록 업계 표준에 맞는 문서화와 모듈식 설계를 갖추어야 한다.
- **Portability**: 앱은 안드로이드 12.0 및 iOS 15.0 이상에서 일관된 사용자 경험을 제공해야 한다.

- **Localization**: 기본적으로 한국어 및 영어를 지원하며 향후 다른 언어도 추가할 수 있는 구조를 갖추어야 한다.
- **Testability**: 모든 핵심 기능은 코드의 최소 80%를 검증할 수 있는 자동화된 테스트 케이스를 갖추어야 한다.
- **Adaptability**: 앱의 대대적인 구조 변경 없이도 변화하는 요구 사항에 맞게 조절할 수 있는 모듈식 구성을 갖추어야 한다.
- **Accessibility:** 적절한 색상 대비, 글자 크기 조절, 화면 읽기 프로그램 지원 등 WCAG 2.1 AA 표준을 준수해야 한다.

3.3.5 Business Rules

- **Basic Qualification**: 사용자는 특정 목표를 달성해야 배치를 얻을 수 있다 (예: “급식 챔피언” 배지는 급식소 10회 방문 시 획득)
- **Pet Sitting Eligibility**: 신뢰도 레벨 2 이상을 달성한 사용자만 펫시팅 서비스를 제공할 수 있다.
- **Review System**: 모든 사용자는 펫시팅 활동 완료 후 48시간 이내에 상대방에 대한 리뷰를 작성해야 한다.
- **Content Moderation**: 모든 커뮤니티 콘텐츠는 커뮤니티 가이드라인을 준수해야 하며, 부적절한 내용은 자동 필터링되고 신고된 게시물은 수동으로 검토되어야 한다.
- **Activity Verification**: 봉사 활동은 해당 위치의 저장된 범위 내에서 사진 인증이 필요하다
- **Donation Allocation**: 플랫폼을 통한 모금된 기부금은 다음과 같이 투명하게 배분되어야 한다: 동물 돌봄 용품 70%, 운영 비용: 20%, 응급 의료 기금 10%.
- **Inactive Account Management**: 6개월 이상 활동이 없는 계정은 휴면 상태로 표시되며, 12개월 이상 비활동 시 데이터는 별도 보관된다.

3.3.6 Community Guideline

Catpus은 캠퍼스 내 동물 돌봄 활동과 사용자 간 상호작용을 위한 안전하고 건강한 커뮤니티 환경을 제공하기 위해 다음과 같은 커뮤니티 가이드라인을 수립한다.

3.3.6.1 가이드라인 목적

본 가이드라인은 Catpus 커뮤니티 내에서 모든 사용자가 존중받고, 동물 복지가 우선시되며, 신뢰할 수 있는 정보가 공유되는 환경을 조성하기 위해 마련되었다. 모든 사용자는 플랫폼 사용 시 이 가이드라인을 준수해야 한다..

3.3.6.2. 콘텐츠 관련 규정

1) 허용 콘텐츠

- 동물 돌봄 및 복지에 관한 정보와 경험
- 캠퍼스 내 길고양이 목격 정보 및 건강 상태 업데이트
- 반려동물 사진 및 일상 공유
- 돌봄 활동 정보 및 참여 독려
- 교육적 가치가 있는 동물 관련 정보

2) 금지 콘텐츠

- 동물 학대 또는 위험한 상황을 묘사하는 콘텐츠
- 불법적인 활동을 조장하거나 홍보하는 내용
- 타인을 비방, 모욕하거나 차별적인 내용
- 허위 정보나 검증되지 않은 의학적 조언
- 지나치게 폭력적이거나 노골적인 내용

3.3.6.3 사용자 에티켓

- 모든 사용자는 서로 다른 의견과 관점을 존중해야 한다.
- 논쟁이 발생할 경우 예의를 지키고 건설적인 방식으로 의견을 교환해야 한다.
- 커뮤니티 내 다양성을 인정하고 포용적인 언어를 사용해야 한다.
- 다른 사용자의 기여와 노력을 인정하고 격려하는 문화를 조성한다.
- 비판은 건설적이어야 하며, 개인이 아닌 행동이나 아이디어에 초점을 맞춰야 한다.

3.3.6.4 동물 복지 및 윤리

- 동물의 안전과 복지가 항상 최우선시되어야 한다.
- 동물 건강에 관한 비상 상황은 즉시 전문가에게 의뢰해야 하며, 플랫폼 내 비상 연락망을 활용할 수 있다.
- 동물을 스트레스나 위험에 노출시키는 행위는 금지된다.
- 동물 돌봄 활동 중 발견한 건강 이상이나 우려사항은 즉시 커뮤니티에 공유해야 한다.
- 모든 돌봄 활동은 가이드라인에 따라 안전하게 수행되어야 하며, 동물의 자연스러운 행동과 습성을 존중해야 한다다.

3.3.6.5 개인정보 보호

- 타인의 개인정보(실명, 연락처, 정확한 거주지 등)를 동의 없이 공개해서는 안 된다.
- 사진이나 영상에 타인이 의도치 않게 포함된 경우, 얼굴이나 식별 가능한 정보를 모자이크 처리해야 한다.
- 특히 아동이 포함된 콘텐츠는 더욱 주의해야 하며, 보호자의 동의 없이 공유해서는 안 된다.
- 위치 정보 공유 시 정확한 주소가 아닌 대략적인 위치로 표시하여 프라이버시를 보호한다.

3.3.6.6 지적재산권

- 타인의 콘텐츠(사진, 글, 영상 등)를 사용할 때는 출처를 명시하거나 허가를 받아야 한다.
- 허가 없이 타인의 콘텐츠를 자신의 것처럼 게시하는 행위는 금지된다.

- 교육 목적의 자료 공유는 허용되나, 저작권법을 준수해야 한다.
- 플랫폼에 게시된 콘텐츠의 무단 외부 유출 및 상업적 활용은 금지된다.

3.3.6.7 가이드라인 위반 시 조치

1) 위반 수준별 대응

- **1단계 위반**: 경고 메시지 발송 및 해당 콘텐츠 삭제
- **2단계 위반**: 일시적 기능 제한(3일~7일간 게시 권한 중지)
- **3단계 위반**: 계정 활동 정지(15일~30일)
- **심각한 위반 또는 반복적 위반**: 영구 계정 정지 및 서비스 이용 제한

2) 이의 제기

- 사용자는 제재 조치에 대해 이의를 제기할 수 있으며, 관리자는 7일 이내에 검토 결과를 통보해야 한다.
- 이의 제기 시 충분한 근거와 설명을 제공해야 한다.

3.3.6.8 신고 및 모니터링

- 모든 사용자는 가이드라인을 위반하는 콘텐츠를 발견할 경우 신고할 책임이 있다.
- 신고는 비밀리에 처리되며, 신고자의 정보는 보호된다.
- 시스템은 금지 키워드 및 패턴을 자동으로 감지하는 필터링 시스템을 갖추고 있다.
- 관리자는 정기적으로 커뮤니티 활동을 모니터링하며, 신고된 콘텐츠는 24시간 이내에 검토된다.
- 허위 신고나 악의적인 신고는 그 자체로 가이드라인 위반으로 간주될 수 있다.

3.3.6.9 가이드라인의 업데이트

- 본 가이드라인은 커뮤니티의 발전과 변화에 따라 정기적으로 검토되고 업데이트 된다.
- 중요한 변경사항이 있을 경우 사용자에게 사전 공지하며, 피드백을 수렴하는 과정을 거친다.
- 최신 가이드라인은 앱 내 설정 메뉴에서 언제든지 확인할 수 있다.

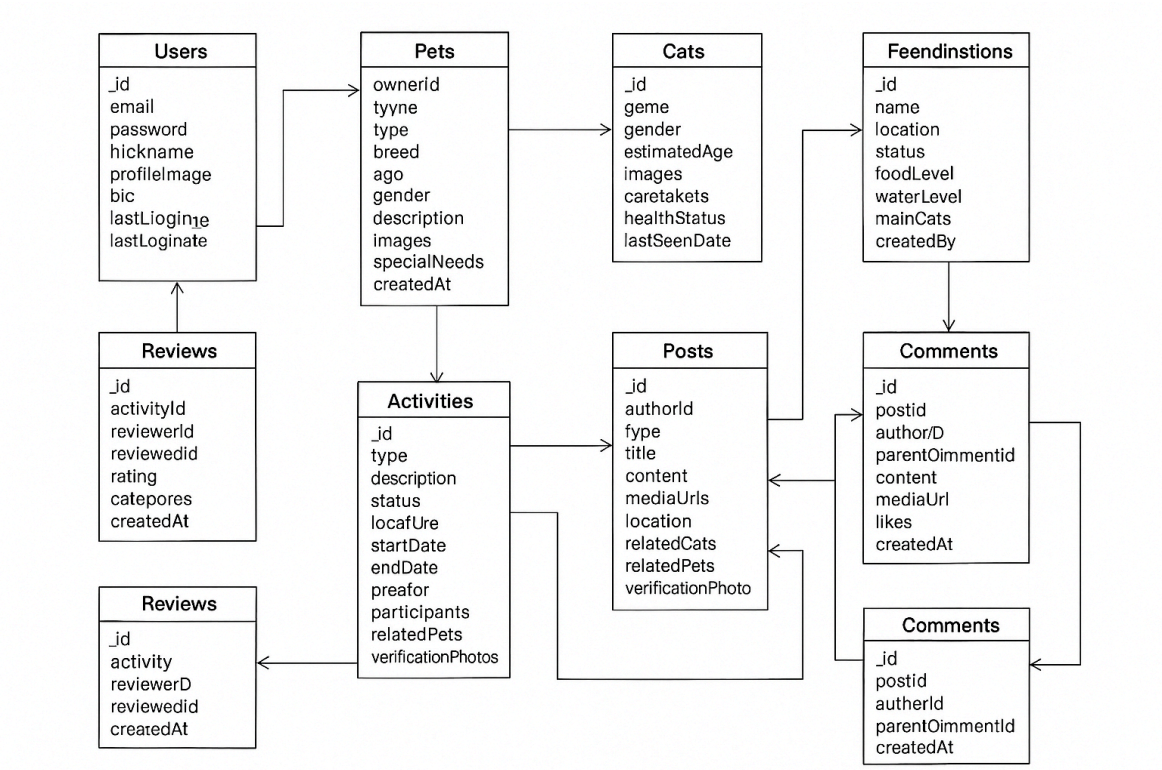
3.4 Other Requirements

3.4.1 Database Requirements

3.4.1.1 데이터베이스 선택 및 구성

- MongoDB를 선택한 이유: MongoDB는 문서 기반 NoSQL 데이터베이스이며 유연한 스키마 및 인덱싱을 제공하여 위치 기반 서비스와 사용자 중심 콘텐츠를 다루는 Catpus에 적합하다.
- 데이터베이스 클러스터 구성: 높은 가용성을 제공하기 위해 3개 노드의 ReplicaSet로 구성한다. 만약 장애가 발생하면 Failover가 이루어지도록 설정한다.

3.4.1.2 ER Diagram



[Figure 11] MongoDB ERD

- **Users - Pets: 1**

  Relationship: 한 사용자는 여러 반려동물을 등록할 수 있다.

- **Users - Badges: M**

  Relationship: 사용자는 여러 배지를 획득할 수 있고, 각 배지는 여러 사용자에게 부여될 수 있다.

- **Users - Activities (as Creator): 1**

  Relationship: 사용자는 여러 활동을 생성할 수 있다.

- **Users - Activities (as Participants): M**

  Relationship: 사용자는 여러 활동에 참여할 수 있고, 각 활동은 여러 참가자를 가질 수 있다.

- **Users - Reviews (as Reviewer): 1**

  Relationship: 사용자는 여러 리뷰를 작성할 수 있다.

- **Users - Reviews (as Reviewee): 1**

  Relationship: 사용자는 여러 리뷰를 받을 수 있다.

- **Activities - Pets: M**

  Relationship: 활동(특히 펫시팅)은 여러 반려동물과 관련될 수 있다.

- **Activities - Cats: M**

  Relationship: 길고양이 관련 활동은 여러 고양이와 관련될 수 있다.

- **FeedingStations - Cats: 1**

  Relationship: 급식소는 여러 주요 고양이와 관련될 수 있다.

- **Users - Posts: 1**

  Relationship: 사용자는 여러 게시물을 작성할 수 있다.

- **Posts - Comments: 1**

  Relationship: 게시물은 여러 댓글을 가질 수 있다.

- **Comments - Comments: 1**

  Relationship: 댓글은 여러 대댓글을 가질 수 있다.

3.4.1.3 데이터 모델 및 컬렉션 구조

아래는 총 10개의 MongoDB 컬렉션 정보이다.

[Table 7] Table of MongoDB Collection Information

1. Users 컬렉션	{
	"_id": ObjectId,
	"email": String,
	"password": String, *// 해시된 비밀번호*
	"nickname": String,
	"profileImage": String, *// 이미지 경로*
	"bio": String,
	"contactInfo": {
	"phone": String,
	"emergencyContact": String
	},
	"preferredAnimals": [String], *// 선호하는 동물 종류 (고양이, 강아지 등)*
	"trustLevel": Number, *// 0-5*
	"trustScore": Number, *// 상세 신뢰도 점수*
	"contributionPoints": Number, *// 기여도 포인트*
	"badges": [
	{
	"badgeId": ObjectId, *// Badge 컬렉션 참조*
	"acquiredDate": Date
	}
	],
	"lastLoginDate": Date,
	"createdAt": Date,
	"isActive": Boolean,

	"role": String, *// 일반 사용자, 관리자 등*
	"settings": {
	"notifications": Boolean,
	"privateProfile": Boolean
	},
	"location": {
	"coordinates": [Number, Number], *// [경도, 위도]*
	"campus": String *// "자연과학캠퍼스", "인문사회캠퍼스" 등*
	},
	"deviceToken": String *// 푸시 알림용 토큰*
	}

2. Pets 컬렉션	{ "_id": ObjectId, "ownerId": ObjectId, *// Users 컬렉션 참조* "name": String, "type": String, *// "고양이", "강아지" 등* "breed": String, "age": Number, "gender": String, "description": String, "images": [String], *// 이미지 경로 배열* "specialNeeds": String, "medicalConditions": [String], "vaccinations": [{ "name": String,

	"date": Date, "expiryDate": Date }], "createdAt": Date, "updatedAt": Date, "isActive": Boolean }

3. FeedingStations 컬렉션	json
	{
	"_id": ObjectId,
	"name": String,
	"description": String,
	"location": {
	"type": "Point",
	"coordinates": [Number, Number] *// [경도, 위도]*
	},
	"campus": String, *// "자연과학캠퍼스", "인문사회캠퍼스" 등*
	"building": String, *// 가까운 건물*
	"status": String, *// "활성", "비활성", "점검 중"*
	"lastFilledDate": Date,
	"foodLevel": Number, *// 0-100%*
	"waterLevel": Number, *// 0-100%*
	"mainCats": [ObjectId], *// Cats 컬렉션 참조*
	"photos": [String],
	"createdBy": ObjectId, *// Users 컬렉션 참조*
	"createdAt": Date,
	"updatedAt": Date,

	"maintenanceSchedule": [
	{
	"day": String, *// "월", "화" 등*
	"time": String,
	"assignedUsers": [ObjectId] *// Users 컬렉션 참조*
	}
	]
	}

4. Cats 컬렉션	json
	{
	"_id": ObjectId,
	"name": String,
	"nickname": String,
	"description": String,
	"gender": String,
	"estimatedAge": String,
	"color": String,
	"neutered": Boolean,
	"identifiers": String, *// 특별한 표식*
	"images": [String],
	"primaryLocation": {
	"type": "Point",
	"coordinates": [Number, Number]
	},
	"frequentLocations": [
	{
	"name": String,
	"coordinates": [Number, Number]
	}

	],
	"healthStatus": String,
	"lastSeenDate": Date,
	"caretakers": [ObjectId], *// Users 컬렉션 참조*
	"medicalHistory": [
	{
	"date": Date,
	"description": String,
	"reportedBy": ObjectId *// Users 컬렉션 참조*
	}
	],
	"createdAt": Date,
	"updatedAt": Date
	}

5. Activities 컬렉션	json
	{
	"_id": ObjectId,
	"type": String, *// "급식소 관리", "펫시팅", "보호소 봉사" 등*
	"title": String,
	"description": String,
	"status": String, *// "생성됨", "신청 중", "매칭됨", "진행 중", "완료됨", "취소됨"*
	"location": {
	"type": "Point",
	"coordinates": [Number, Number]
	},
	"address": String,
	"startDate": Date,
	"endDate": Date,

	"requiredParticipants": Number,
	"creator": ObjectId, *// Users 컬렉션 참조*
	"participants": [
	{
	"userId": ObjectId, *// Users 컬렉션 참조*
	"status": String, *// "수락됨", "대기 중", "거절됨"*
	"joinedAt": Date
	}
	],
	"relatedPets": [ObjectId], *// Pets 컬렉션 참조 (펫시팅 경우)*
	"relatedCats": [ObjectId], *// Cats 컬렉션 참조 (길고양이 관련 활동)*
	"relatedFeedingStation": ObjectId, *// FeedingStations 컬렉션 참조*
	"verificationPhotos": [
	{
	"url": String,
	"uploadedBy": ObjectId, *// Users 컬렉션 참조*
	"uploadedAt": Date,
	"location": {
	"coordinates": [Number, Number]
	}
	}
	],
	"completionNotes": String,
	"createdAt": Date,
	"updatedAt": Date
	}

6. Reviews 컬렉션	json
	{
	"_id": ObjectId,
	"activityId": ObjectId, *// Activities 컬렉션 참조*
	"reviewerId": ObjectId, *// Users 컬렉션 참조*
	"revieweeId": ObjectId, *// Users 컬렉션 참조*
	"rating": Number, *// 1-5*
	"comment": String,
	"categories": [String], *// "친절함", "전문성", "시간 준수" 등*
	"createdAt": Date,
	"updatedAt": Date,
	"isPublic": Boolean
	}

7. Posts 컬렉션	json
	{
	"_id": ObjectId,
	"authorId": ObjectId, *// Users 컬렉션 참조*
	"type": String, *// "Petstagram", "커뮤니티 게시글", "공지사항"*
	"title": String,
	"content": String,
	"mediaUrls": [String],
	"location": {
	"type": "Point",
	"coordinates": [Number, Number]
	},
	"tags": [String],
	"mentions": [ObjectId], *// Users 컬렉션 참조*
	"relatedCats": [ObjectId], *// Cats 컬렉션 참조*

	"relatedPets": [ObjectId], *// Pets 컬렉션 참조* "likes": [ObjectId], *// Users 컬렉션 참조* "commentsCount": Number, "isEdited": Boolean, "isPinned": Boolean, "visibility": String, *// "공개", "친구만", "비공개"* "createdAt": Date, "updatedAt": Date }

8. Comments 컬렉션	json
	{
	"_id": ObjectId,
	"postId": ObjectId, *// Posts 컬렉션 참조*
	"authorId": ObjectId, *// Users 컬렉션 참조*
	"parentCommentId": ObjectId, *// 자기 참조 (대댓글 용)*
	"content": String,
	"mediaUrl": String,
	"likes": [ObjectId], *// Users 컬렉션 참조*
	"isEdited": Boolean,
	"createdAt": Date,
	"updatedAt": Date
	}

9. Badges 컬렉션	json { "_id": ObjectId, "name": String, "description": String,

	"iconUrl": String,
	"category": String, *// "봉사", "펫시팅", "커뮤니티" 등*
	"requirements": {
	"type": String, *// "활동 횟수", "평점", "연속 활동" 등*
	"value": Number,
	"additionalConditions": String
	},
	"tier": Number, *// 1, 2, 3 등 난이도*
	"createdAt": Date,
	"updatedAt": Date,
	"isActive": Boolean
	}

10. Notifications 컬렉션	json
	{
	"_id": ObjectId,
	"recipientId": ObjectId, *// Users 컬렉션 참조*
	"senderId": ObjectId, *// Users 컬렉션 참조 (옵션)*
	"type": String, *// "매칭", "리뷰", "댓글", "급식소 알림", "배지 획득"*
	"title": String,
	"content": String,
	"relatedObjectId": ObjectId, *// 관련 객체의 ID*
	"relatedObjectType": String, *// 관련 객체의 컬렉션 이름*
	"isRead": Boolean,
	"createdAt": Date,
	"expiresAt": Date,
	"actionUrl": String *// 알림 클릭 시 이동할 경로*
	}

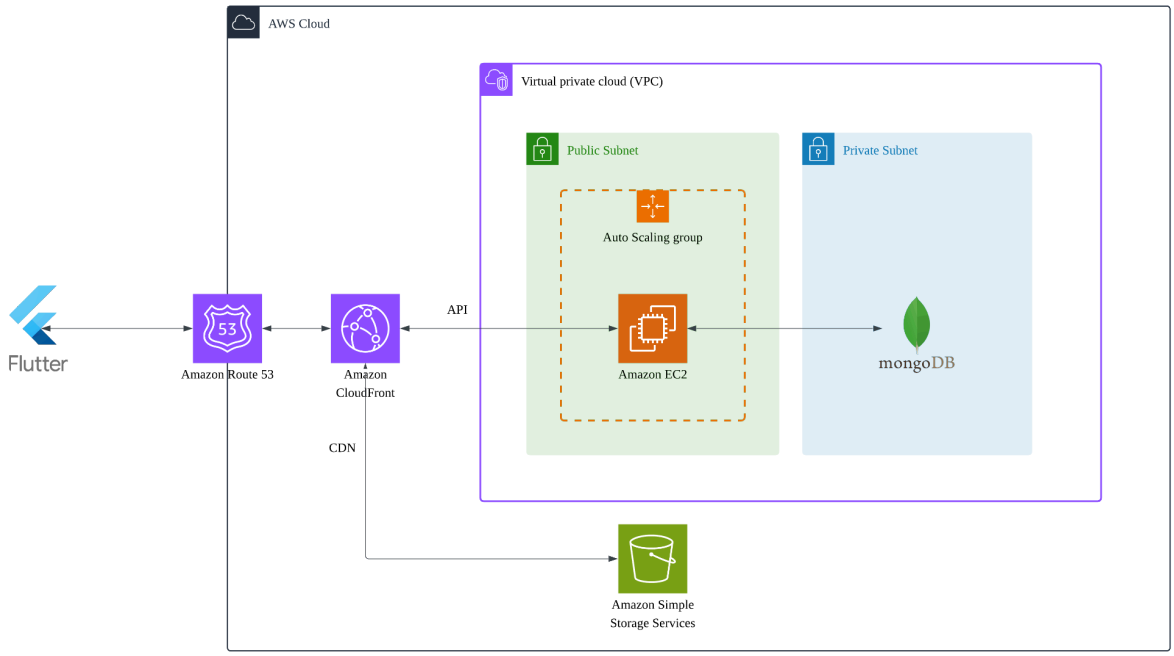
3.4.1.4 인덱싱 전략

- 사용자 검색 최적화
  - User.email: 유니크 인덱스
  - Users.nickname: 텍스트 인덱스
  - User.trustLevel: 일반 인덱스
- 활동 내역 조회 최적화:
  - Activities.participants.userId: 복합 인덱스
  - Activities.status: 일반 인덱스
  - Activities.startDate: 일반 인덱스
  - 복합 인덱스: Activities.type, Activities.status, Activities.startDate
- 게시물 및 댓글 최적화:
  - Posts.authorId: 일반 인덱스
  - Posts.createdAt: 일반 인덱스
  - Posts.tags: 다중키 인덱스
  - Comments.postId: 일반 인덱스

3.4.1.5 데이터 제약 조건

- 유니크 제약 조건:
  - User.email: 이메일 중복 방지
  - Badges,name: 배지 이름 중복 방지
  - 복합 유니크 인덱스: Reviews.activityId, Reviews.reviewerId, Reviews.revieweeId -> 동일 활동에 대한 중복 리뷰 방지
- 값 범위 제약:
  - Reviews.rating: 1~5 범위 검증
  - Users.trustLevel: 0~5 범위 검증
  - FeedingStations.foodLevel, FeedingStations.waterLevel: 0~100 범위 검증

3.4.2 System Architecture



[Figure 12] System Architecture

1. 클라이언트 및 프론트엔드는 Flutter를 프레임워크로 사용하여 iOS와 Android 앱을 하나의 코드 베이스로 개발
2. Amazon Route 53을 사용해 DNS 웹 서비스로 사용자 요청을 적절한 AWS 리소스로 라우팅
3. Amazon Cloudfront의 CDN 서비스로 사용자에게 더 빠른 콘텐츠 전송 및 캐싱을 할 수 있게 도와준다.
4. Amazon EC2가 Auto Scaling Group 내에서 구성되어 있으며, 트래픽 변화에 따라 자동으로 확장/축소 된다. 이를 통해 사용자가 많은 시간대에도 안정적인 서비스를 제공한다.
5. MongoDB는 Private Subnet에 위치하여 직접적인 외부 접근을 차단하고 앱 서버만 DB에 접근할 수 있도록 설계되었다.

3.4.2.1 사용자 인증 및 계정 관리

1. Node.js 서버는 사용자 인증 정보를 MongoDB에 저장하고 검증한다.
2. JWT 토큰 기반 인증을 사용하며, Refresh Token은 MongoDB에 저장된다.

3.4.2.2 급식소 관리 및 위치 서비스

1. 급식소 사진은 Amazon S3에 저장되며, Cloudfront를 통해 빠르게 전송된다.

3.4.2.3 펫시팅 매칭 시스템

1. 펫시팅 요청 및 수락은 Node.js 서버를 통해 처리되며, MongoDB에 매칭 데이터가 저장된다.

3.4.3 Organization Requirements

3.4.3.1 Environmental Requirements

- **"Catpus"** 는 안드로이드 및 iOS 운영체제를 지원하는 모바일 앱으로 제공된다.
  - 지원하는 운영체제 버전은 다음과 같다.
  - Android OS 12.0 (API 레벨 31) 이상
  - iOS 15.0 이상
- 앱을 사용하기 위해 다음 권한에 대한 허용이 필요하다
  - 카메라 및 갤러리 접근 권한 (사진 업로드용)
  - 위치 정보 접근 권한 (위치 인증 및 지도 기능용)
  - 푸시 알림 권한 (실시간 알림 수신용)
  - 저장소 접근 권한 (데이터 캐싱 및 저장용)
- **성균관대학교 자연과학캠퍼스 내에서의 활동을 전제로 하며, 캠퍼스 내 위치 정보가 정확히 매핑되어야 한다.**

3.4.3.2 Development Requirements

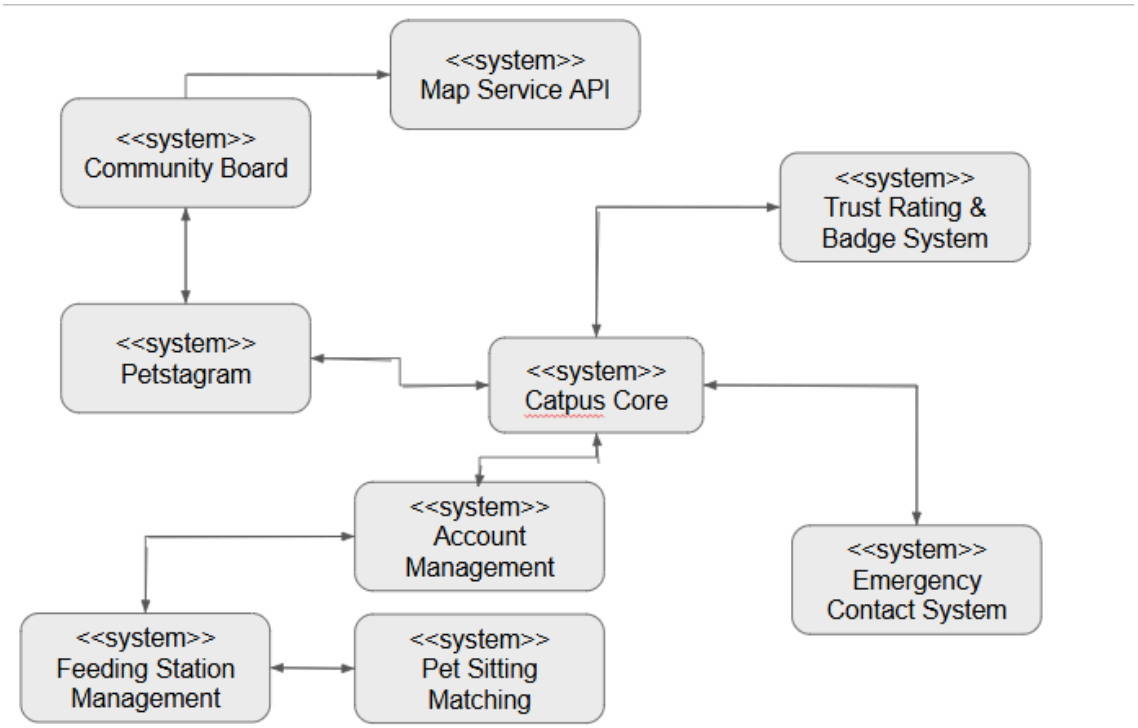
- 개발 환경 표준화:
  - 프론트엔드: Flutter (0.7.0 이상)
  - 백엔드: Node.js(16.0.0 이상), Express.js(4.18.0 이상)
  - 데이터베이스: MongoDB(5.0.0 이상)
- 버전관리 및 협업:

    - Git을 활용한 버전 관리
    - Github Flow 브랜치 전략 채택
    - Jira를 활용한 작업 관리
- 빌드 및 배포 자동화:
    - CI/CD 파이프라인 구축 (Github Actions 활용)
    - 개발, 프로덕션 환경 분리

3.4.3.3 Regulatory Requirements

- **개인정보 보호법 준수:**
    - 개인정보 수집 시 명시적 동의 획득
    - 수집된 개인정보의 목적 외 사용 금지
    - 개인정보 보관 기간 설정 미 만료 시 자동 파기
    - 개인정보처리방침 명시 및 변경 시 사전 고지
- **동물보호법 관련 규정:**
    - 동물학대 관련 신고 및 처리 체계 마련
    - 길고양이 돌봄 활동 시 동물보호법 준수 안내
- **학내 규정 준수:**
    - 성균관대학교 시설 이용 규정 준수
    - 캠퍼스 내 동물 관련 활동에 대한 학교 방침 준수
    - 학생 단체 활동 관련 규정 준수

3.4.3.4 Context Model



[Figure 13] Context Model

4. Appendix

4.1 Document History

[Table 8] Table of Document History

Name	Date	Reason For Changes	Version
전체	2025.05.07	문서작업시작	0.0.0
최민성	2025.05.07	1.1~1.5 작성	0.1.2
박보연	2025.05.07	2.1~2.3 작성	0.1.3
오정호	2025.05.08	2.4~2.7 작성	0.2.1
박진석	2025.05.08	3.3~3.4 작성 / E/R, dataflow, use case	0.2.3
조현우	2025.05.09	3.1~3.3 작성 / User Interface 추가	0.2.4
박보연	2025.05.10	User Interface 그림 추가	1.0.0
박진석	2025.05.10	Database 작성	1.0.1
박진석	2025.05.10	Context Model 작성	1.0.2
최민성	2025.05.10	1.2 Definition으로 수정	2.0.0
박지수	2025.05.10	3.2 작성	2.0.1
박지수	2025.05.10	Use Case Diagram 작성	2.0.2
박보연	2025.05.11	3.1.1 표 형식 재구성	2.1.1
박보연/오정호/최민성	2025.05.11	Table, Figure 레이블링 및 페이지 번호 수정	2.1.2
박진석	2025.05.11	용어 통일/오타 수정	2.1.3
전체	2025.05.11	최종 서식 수정	2.1.4